Where Does Hypergraph Mapping Pay Off? Mapping EEG Spiking Neural Networks from Seizure Detection and Motor Imagery onto Neuromorphic Cores

Rishav Singh
Dept. of ECE (AI&ML)
MIT World Peace University
Pune, India

Dr. Sarika Patil
Dept. of ECE
MIT World Peace University
Pune, India

***Abstract*—Spiking neural networks (SNNs) can run on neuromorphic chips with very little energy. But a trained network cannot just be dropped onto a chip. Its neurons must first be split into groups, and each group must be placed on a core. A poor split sends many spikes between cores, and that costs energy and time. We study this mapping step for EEG SNNs on two tasks: seizure detection (CHB-MIT, patient chb01) and four-class motor imagery (BCI Competition IV-2a). We describe each network as a hypergraph, with one hyperedge per sending neuron and measured firing rates as weights. We compare a hypergraph-aware pipeline with a naive layer-order pipeline under a Loihi-based cost model. For seizure detection, five models passed an accuracy gate. The energy-latency product (ELP) fell by 23 to 73 percent. Most of this came from placement. Partitioning was clear only on the large sparse network, about 11 percent. For motor imagery, we trained nine models at 50 percent sparsity. The full pipeline beat the naive one in 9 of 9, lowering ELP by about 28 to 62 percent, more on larger networks. Overlap partitioning added 15 to 31 percent beyond placement. Mapping did not change accuracy. Hardware checks at four levels showed that the gain survives trace replay and cycle-level simulation, but a single front-end input port removes it. Int8 did not pass our declared gate. Sparsity at 50 percent saved almost no real memory once index storage was counted. No mapping fits the assumed per-core limits. We give a plain answer to when mapping pays off, and we report the lessons and the ideas that failed.**

***Index Terms*—spiking neural networks, neuromorphic hardware, hypergraph partitioning, core mapping, EEG, seizure detection, motor imagery**

I. Introduction

Seizures can be found in scalp EEG, and many deep learning models do this well. Most of them use a lot of power, which makes them hard to run all day on a small device. Spiking neural networks (SNNs) are another option. A neuron in an SNN sends a spike only when something happens, so a chip built for spikes can save a lot of energy. Busia *et al.* [1] showed that a very small SNN on an FPGA can detect seizures in CHB-MIT recordings [2], [3] at a tiny energy cost per inference. Zhang *et al.* [4] reported a recurrent spiking convolutional network for cross-patient seizure detection. Their theoretical energy estimate is far below that of conventional neural networks.

This project started as an attempt to build on that wearable detector. Our first idea was to group EEG channels with hypergraphs before the spike encoder [5]. Our second idea was to build a small FPGA system around the result. Neither idea survived testing, and we describe both in Section IV. What was left was a more basic question: once an SNN is trained, how should its neurons be placed on a chip that is made of many small cores?

A neuromorphic chip has many cores. Each core can hold only a limited number of neurons, inbound axons, and synapses. When a spike must go from one core to another, it travels over an on-chip network, and this costs energy and time. So the place where each neuron lives matters. Ronzani and Silvano [6] point out that a neuron sends the same spike to all of its targets. If five targets sit in one core, the chip sends one message and the core copies it locally. A normal graph does not see this and counts five messages. They therefore model the SNN as a hypergraph, with one hyperedge for each sending neuron. The same authors later reported a GPU-based flow with multi-level partitioning and swap-based placement. In their model-based tests it gave up to 42 percent lower energy and 16 percent lower latency than sequential tools, and mapping was 18 to 280 times faster [7]. It uses the same Loihi-based cost constants as ours.

We built a mapping pipeline around this idea and first tested it on seizure-detecting SNNs of three sizes. We then added a second EEG task, four-class motor imagery from BCI Competition IV [8], dataset 2a [9], with nine new SNNs. The paper is now about one question: when does hypergraph-aware mapping pay off for EEG SNNs, across two tasks? We did not only look for a win. We also asked where any gain comes from, checked it with hardware-style tests, and kept a record of every idea that failed. Accuracy is a gate and a check in this work, not the goal. A large part of this paper is about what we concluded at each step and how that changed what we did next.

The main contributions are:

- A hypergraph-aware mapping pipeline, compared with a

naive layer-order pipeline on seizure and motor-imagery SNNs.
- Ablations that split the gain into a placement part and a partitioning part, on both tasks.
- A nine-model motor-imagery study with dense, pre-sparse and post-sparse training.
- Hardware checks at four levels (formula, trace replay, cycle-level mesh, and RTL counters), plus int8, memory and front-end traffic checks.
- A full record of failed ideas and the lessons we drew from them.

Section II gives background. Section III describes the data and the baseline models. Section IV covers the early ideas that did not hold up. Section V explains the mapping method, and Section VI the models and setup. Section VII gives the mapping results and Section VIII the hardware checks. Section IX gives the lessons, and Sections X and XI the limits and the conclusion.

II. Background and Related Work

A. SNNs for Seizure Detection

Maass called SNNs the third generation of neural network models [10]. Roy *et al.* reviewed spike-based machine intelligence with neuromorphic computing [11]. Nunes *et al.* wrote a survey of SNNs [12]. An SNN neuron keeps an internal value that leaks over time. Input spikes raise the value. When the value crosses a threshold, the neuron fires and the value is reduced. We use the standard leaky integrate-and-fire (LIF) neuron. For the first replication we used snnTorch [13], and for the later work we wrote our own PyTorch code.

Busia *et al.* [1] built a seizure detector on a Lattice iCE40UP5K FPGA. It reads four temporal EEG channels. Each sample is turned into one spike on one of 16 amplitude levels, so the input layer has 64 neurons and exactly four of them are active at any step. A 64-8-1 LIF network with 529 weights makes the decision. They report 4.55 nJ and 0.5 μs per inference. We use their encoder and network shape as our starting point. Zhang *et al.* [4] used a recurrent spiking convolutional network for cross-patient detection. They estimate its energy in theory and not on a chip.

B. SNNs for Motor Imagery

In motor imagery, a person imagines a movement without doing it, and the EEG is used to tell which movement. Schirrmeister *et al.* studied convolutional networks for EEG decoding and visualization [14]. Lawhern *et al.* proposed EEGNet, a small convolutional network that uses depthwise separable convolutions [15]. Our front end also uses a depthwise convolution, but our network is not EEGNet.

Several groups used SNNs for this task. Kumar *et al.* studied EEG decoding with SNNs on neuromorphic hardware [16]. Tan *et al.* spike-encode filtered EEG with BSA [17], a spike-train encoding scheme [18]. They classify with a convolutional LIF network built in snnTorch, and report 82.08 percent average accuracy on BCI IV-2a. We do not use BSA. Banovoth and Kadambari picked key EEG channels from a correlation graph with Roman domination, used BSA encoding, and reported 81.75 percent on IV-2a [19]. They report that channel selection helped, while our hyperedge grouping on seizure data did not. Ma *et al.* proposed DRCANet, a spiking convolutional network with attention, trained by ANN-to-SNN conversion. It reaches 83.65 percent on IV-2a [20]. Its energy estimate counts multiply-accumulate and accumulate operations (about 56 percent below the conventional network), not on-chip communication. Its output layer sums the membrane potential over time, like our readout. Liao *et al.* proposed SCNet, a convolutional SNN with learnable coding, which is quoted at 67.43 percent on four-class IV-2a in later work [21]. The same group built LENet, a lightweight SNN that replaces the fully connected layer by a classification convolution [22]. Li *et al.* proposed HR-SNN for four-class motor imagery, quoted at about 77.6 percent on IV-2a in later work [23]. Virgilio *et al.* applied SNNs to the classification of motor tasks in EEG [24]. Kasabov proposed NeuCube, an STDP-based SNN that suits shallow networks [25]. These papers focus on accuracy. The energy estimate of Ma *et al.* counts operations and not communication between cores.

C. Mapping SNNs onto Cores

Chips such as Loihi [26], TrueNorth [27] and SpiNNaker [28] are built from many cores joined by a network-on-chip (NoC). Kudithipudi *et al.* reviewed neuromorphic computing at scale [29]. Mapping has two steps. *Partitioning* decides which neurons share a core. *Placement* decides where each core sits on the chip grid. Each core has three limits: the number of neurons (C_{npc}), the number of distinct inbound axons (C_{apc}), and the number of synapses (C_{spc}) [6].

Ronzani and Silvano [6] model the SNN as a directed hypergraph. Two ideas follow. *Synaptic reuse* says that neurons which listen to the same senders should share a core, so the core receives each spike once. *Connections locality* says that cores which talk a lot should sit close together on the grid. They also give a greedy partitioner based on hyperedge overlap. Their GPU-based follow-up [7] uses the same cost constants as we do.

Other mapping work is close to ours. Balaji *et al.* built SpiNeMap, which clusters and then places, and reported 45 percent less energy and 21 percent less latency than the best earlier technique [30]. Li *et al.* built SNEAP, a toolchain for large SNNs on NoC-based platforms [31]. Xue *et al.* built EdgeMap for edge computing [32]. Jin *et al.* used Hilbert-curve placement with force-directed refinement [33], which is the idea our placement step follows. The same group later mapped large SNNs onto arbitrary meshed hardware [34]. Xing *et al.* proposed SNNcut, which partitions large SNNs using spike sharing [35]. Huynh *et al.* reviewed how SNNs are implemented on neuromorphic architectures [36]. Mt-KaHyPar is a general scalable hypergraph partitioner [37]. Catalyurek *et al.* surveyed recent advances in hypergraph and graph partitioning [38].

TABLE I
DATA SPLIT USED FOR THE FINAL SEIZURE EXPERIMENTS (CHB01).

Set	Records	Windows	Seizure windows
Train	03, 04, 15, 21, 26	2090	40
Validation	16	450	6
Test	18	450	11

D. Trace-Driven Evaluation

Mapping papers such as SpiNeMap [30], SNEAP [31], and EdgeMap [32] do not rely only on formulas. They record real spike traces from the software network and replay them on a cycle-accurate model of the on-chip network. We follow this idea in Section VIII.

E. Hypergraphs for EEG

Pisarchik *et al.* [5] build brain hypergraphs from coherence, cross-correlation, and mutual information. The reason is that one seizure can involve several regions at the same time, and a hyperedge can join them all. We used this as our first hypothesis, at the input side of the SNN (Section IV-A).

III. DATA AND BASELINE MODELS

This section describes the data of both tasks. Fig. 1 shows the overall pipeline.

A. Seizure Data

1) Recordings and splits: We used CHB-MIT [2], [3], patient chb01 only, sampled at 256 Hz. We took the same four bipolar channels as Busia *et al.*: F7-T7, T7-P7, F8-T8, and T8-P8. One practical problem came up early. The recordings contain two channels named T8-P8 because of a montage change, and the reader library renamed them. We fixed this by always picking the original one. After that, the seizure intervals we parsed added up to 442 s over seven records, which matches the total in Busia *et al.* and told us that channel choice and label parsing were correct.

For the final experiments we cut each record into non-overlapping windows of 2048 steps (8 s). A window is labeled as seizure if at least half of it overlaps a seizure. Files were split by record, not by window, so that neighbouring windows could not leak between sets (Table I). We also checked our labels against the official summary file, and all seven seizure intervals matched.

2) Spike encoding: Each channel's amplitude range is cut into 16 equal levels. The range comes from the first 300 s of record 03. The level steps came out as 34.5, 33.2, 32.6, and 28.1 μV for the four channels. At every time step, each channel produces one spike on the level where its voltage falls. This gives 64 input neurons and exactly 4.0 input spikes per step, which we checked in the code.

3) Replicating the wearable detector: We first rebuilt the 64-8-1 network of Busia *et al.* (529 parameters, including 9 biases) and tested it with leave-one-record-out folds. Training on a laptop was far too slow. A full 500-epoch run over all seven folds was projected at about 58 hours. We moved to a GPU and cut training to 10 epochs per fold. We also dropped the paper's post-ictal filtering and output smoothing. One fold (record 18) gave a dead model whose output never fired, and we left it out of the average and reported it separately. Table II shows the other six folds.

TABLE II
REPLICATION ON CHB01 (6 OF 7 FOLDS) AGAINST THE PUBLISHED ROW.

Metric	Ours	Busia *et al.*
Accuracy	0.992	0.994
Sensitivity (segment)	1.000	0.687
Specificity	0.992	1.000
AUC	0.998	0.976
Event sensitivity	1.000	1.000
False positives per hour	1.776	0.000

AUC and event sensitivity match or beat the paper. False positives are much higher, and this is what we expect after removing the smoothing and the post-ictal rule. Our threshold came from a per-fold ROC rule, which is not the paper's rule, so segment sensitivity and specificity cannot be compared one to one. The main point is that a working baseline now existed and everything after this builds on it.

B. Motor-Imagery Data

We used BCI Competition IV dataset 2a [8], [9], loaded through MOABB (`BNCI2014_001`) [39]. It has 9 subjects, 22 EEG channels, a rate of 250 Hz, and four classes: left hand, right hand, feet, and tongue. Jayaram and Barachant found that the same pipeline can give very different results on different datasets [39], so we do not read much into our accuracy on this one dataset.

There are 5,184 trials of shape 22×1001 (1,296 per class), filtered from 8 to 30 Hz. We mixed the two sessions and made one stratified 80/20 split (seed 42). This gives 4,147 training and 1,037 validation trials. We applied Euclidean alignment per subject, which makes trials from different subjects more alike without using labels [40]. After that, each trial was normalised by its standard deviation. As a simple reference, log-variance features with LDA reach 44.9 percent. Chance is 25 percent.

There is no separate test set. The validation trials are used for model choice, for measuring firing rates, and for reporting. We say this again in Section X.

IV. EARLY IDEAS THAT DID NOT HOLD UP

A. Hypergraphs on EEG Channels

Our first hypothesis was that the baseline treats the four channels separately and misses how electrodes move together in a seizure. We tested three setups on chb01, changing only the channel strategy. Everything else stayed the same: encoder, network shape, training, and a split by file. Records 18 and 26 were held out.

- **A:** the four fixed channels of Busia *et al.*
- **B:** four channels picked from all 21 by a single-channel connectivity score.

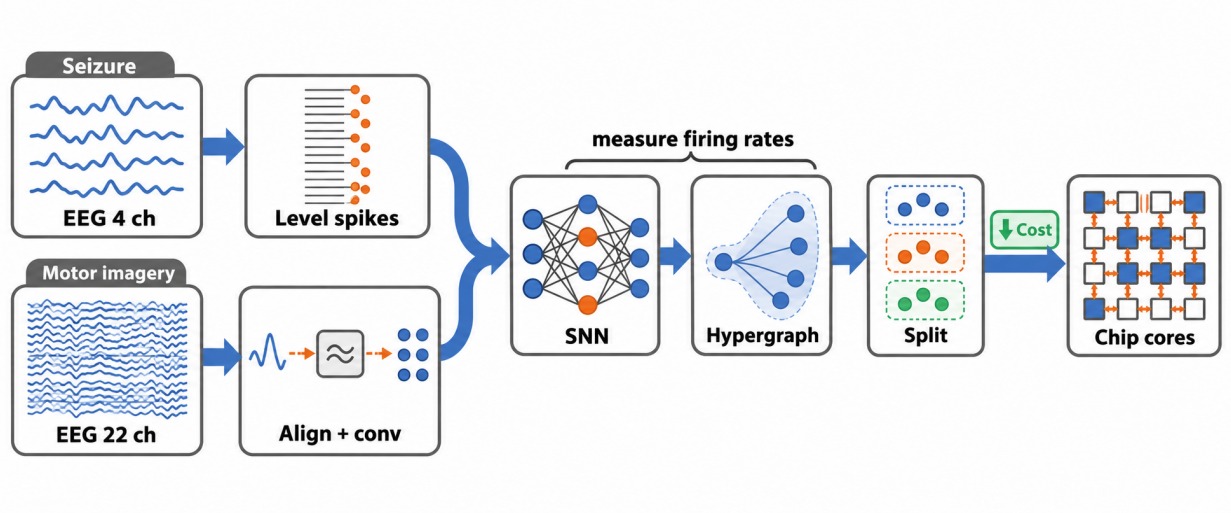


Fig. 1. Overall pipeline for two EEG tasks, from EEG to the estimated cost on chip cores. Seizure lane: four channels and a 16-level one-spike encoder. Motor-imagery lane: 22 channels, alignment and a conv front-end into the first LIF layer. Both lanes feed the same SNN, hypergraph, split and chip-core steps.

TABLE III
EEG-SIDE ABLATION ON CHB01 (ONE SPLIT).

Setup	Sens.	Spec.	Epochs to converge
A: fixed 4 channels	0.989	0.991	9–12
B: data-driven 4 channels	0.968	0.995	9–12
C: hyperedge groups	0.968	0.990	12–15

TABLE IV
FIRST MAPPING TESTS: INTER-CORE MESSAGES (KM1).

Network	Neurons/core	Naive	Hypergraph	Change
Real (73 neurons)	8	144.0	136.0	−5.6%
Real (73 neurons)	4	208.0	266.0	+27.9%
Synthetic (1000)	16	3752.4	2827.7	−24.6%
Synthetic (1000)	8	3911.8	2992.9	−23.5%

- **C:** hyperedge groups. We computed coherence, cross-correlation, and mutual information for all channel pairs, subtracted the resting-state values, kept the strong pairs, joined them into groups, and averaged each group with connectivity weights.

Setup A won (Table III). Setup C was not better than B and needed more epochs. There was also a warning sign in the construction itself. The threshold of the original method (mean plus one standard deviation) gave one giant group of 17 channels. We had to raise it to 1.75 standard deviations by hand to get four small groups. We decided not to tune it further, because tuning on one split would only fit that split. This is a single patient and a single split, so it does not prove that the idea can never work. It does show that the idea is not robustly better, so we dropped it as the main contribution.

B. First Mapping Tests

After this, we moved the hypergraph to the hardware side. We took the trained 73-neuron network (64-8-1) and compared naive sequential partitioning with hypergraph partitioning (Mt-KaHyPar [37], KM1 objective). Because the real network was so small, we also built a synthetic sparse network of 1000 nodes with distance-decaying connections. Table IV gives the number of messages that cross cores.

On the real network the result was small or even negative. We had predicted this before the run. Its first layer is fully dense, so every input connects to all 8 hidden neurons and there is no structure for a smart partitioner to find. On the synthetic sparse network the gain was about 24 percent at both core sizes. So the mechanism works when the network is sparse and local, but our trained network was too small and too dense to show it.

C. The Wearable Target Did Not Fit

A second problem is the wearable story. The design of Busia *et al.* already uses 3271 of the 5280 logic cells (61%) on its FPGA. Its 529 weights fit in one core, so there is nothing to split. A bigger network would give the partitioner more to do, but it would also need more hardware than a wearable chip has. The sample period at 256 Hz is 3.9 ms, while the reported inference time is 0.5 μs, so latency is not the bottleneck either. We therefore stopped treating the wearable as the goal. The question became: at what network size and sparsity does hypergraph mapping start to pay off for an EEG SNN?

D. Results We Later Withdrew

A first full pipeline gave a “90 percent sparse winner” with traffic ratios of 0.10 to 0.58. A careful review showed three problems. The sparse networks were not trained. The baselines never put an input neuron on the same core as its targets. And the sparse mask had planted groups of identical inputs, so

finding them proved little. We withdrew the claim. The one result that held up was a null result: on the trained dense 73-neuron model, a simple rate-aware sequential mapping (traffic 1.48) beat hypergraph overlap partitioning (2.15).

A deeper attempt on a 417-neuron network was also inconclusive. The Mt-KaHyPar call crashed and the code silently fell back to a home-made annealer. The two methods called "EdgeMap" and "Overlap" turned out to be the same code. The headline network had a validation AUC of only 0.83 after 10 optimizer steps. Two useful facts still came out of it. A plain-graph count overestimated spike traffic by 6.9 to 31.5 times compared with the hypergraph count. And spreading the network over 99 cores instead of 14 raised energy from 167 to 287 pJ even though abstract traffic went down. We decided to stop patching this code and rebuild the pipeline from the paper.

The motor-imagery study also had early problems. An old training run did far too few updates, and old result files sat next to new ones. We describe both in Section IX.

V. Mapping Method

A. From a Trained SNN to a Hypergraph

After training, we run real EEG windows through the network and count spikes. The firing rate of a neuron is its spike count divided by the number of time steps and windows. We then build the hypergraph $H = (V, E)$. Each neuron is a node. Each neuron that sends spikes is the source of one hyperedge, and its targets are the destinations given by the connection mask. The weight w_e of a hyperedge is the measured firing rate of its source (Fig. 2).

One point cost us a lot of time early on. Spike frequency is not the synaptic weight. The learned weights decide which neurons are in a hyperedge. The firing rate of the source decides how heavy the hyperedge is. Rates must be measured by running data through the network. They cannot be read from the weight matrix. In the seizure models, input neurons count as nodes, and their spikes count as traffic, so the input rates always sum to 4.0.

We apply the same method to the motor-imagery models. Their hypergraph has the conv units as the first nodes and has no input layer. The readout nodes have rate 0, so they add no traffic.

B. Cost Model

Let λ_e be the number of different cores that hyperedge e touches, counting the core of the source. The connectivity cost is

$$C = \sum_{e \in E} w_e \, (\lambda_e - 1). \tag{1}$$

When a spike travels over d grid hops to another core, the energy and latency of that delivery are

$$E_d = d\,(E_R + E_T) + E_R, \qquad L_d = d\,(L_R + L_T) + L_R, \tag{2}$$

with $E_R = 1.7$ pJ, $E_T = 3.5$ pJ, $L_R = 2.1$ ns, and $L_T = 5.3$ ns. These values come from the Loihi-based table in [6]. The total energy is the weighted sum of E_d over all deliveries. The average latency is the weighted average of L_d. We call their product the energy-latency product (ELP), and lower is better. Every mapping is scored by the same function, so only the method changes and the yardstick does not. These numbers are estimates from a model, not measurements on a chip.

C. Two Pipelines

We compare a naive pipeline with a hypergraph-aware one.

Naive. Neurons fill cores in the order of their IDs, which is layer by layer. Rates and wiring are not used. Cores are placed row by row on the grid.

Hypergraph-aware. Each core starts with the busiest unassigned neuron. We then keep adding the neuron that shares the most inbound hyperedges with the core, until the core holds 32 neurons. This follows the idea of synaptic reuse, but it is a simpler greedy version and not a line-by-line copy of the algorithm in [6]. Cores are placed on the grid along a Hilbert curve. After that, we try 1500 random swaps of core positions and keep a swap only if the ELP gets better.

D. Ablation Design

The two pipelines differ in two ways at once: partitioning and placement. To find out which one matters, we ran four variants. Variant A is naive partitioning with row-by-row placement. B is naive partitioning with Hilbert placement and swap refinement. C is overlap partitioning with row-by-row placement. D is overlap partitioning with Hilbert placement and refinement, which is our full pipeline. The fair test of partitioning is D against B, because both get the same placement effort. We ran each variant over five seeds (42 to 46), since the swap step is random.

VI. Models and Experimental Setup

A. Seizure Models and the Accuracy Gate

1) A ladder of six models: We trained six SNNs in three sizes, each with a dense version and a 90 percent sparse version (Table V). Sparse layers use a fixed fan-in mask, so every neuron keeps the same number of incoming synapses. The neurons are LIF with leak 0.9, threshold 1.0, reset by subtraction, and a fast-sigmoid surrogate gradient. The last unit is a non-spiking leaky integrator that is averaged over time, so it never sends spikes and carries no traffic. All models used the same recipe: Adam with learning rate 2×10^{-3}, batch size 64, five epochs (165 optimizer steps), binary cross-entropy, and gradient clipping at 1.0. The best epoch was picked by validation AUC.

2) Accuracy gate: We decided on a gate before looking at any mapping result. A model is valid only if its output is not dead and both its validation and test AUC are at least 0.95. The threshold comes from the training data by Youden's J. Only valid models are mapped. This rule is there because our earlier mapping results were measured on networks that did not work.

Five models passed. `SZ-S_sparse` failed, and we left it out and did not try to rescue it. At 90 percent sparsity, each of its 8 hidden neurons sees only 6 of the 64 inputs, and 33

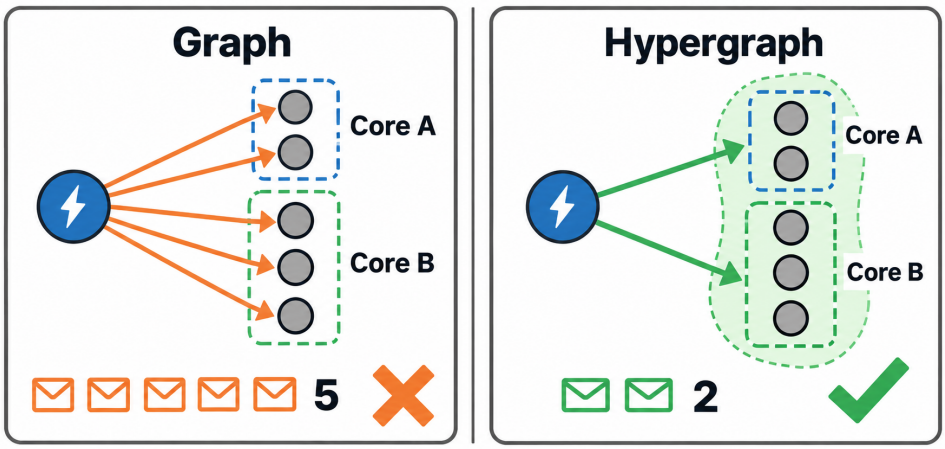


Fig. 2. A plain graph counts every connection as a separate message. A hypergraph counts one message per sending neuron and core.

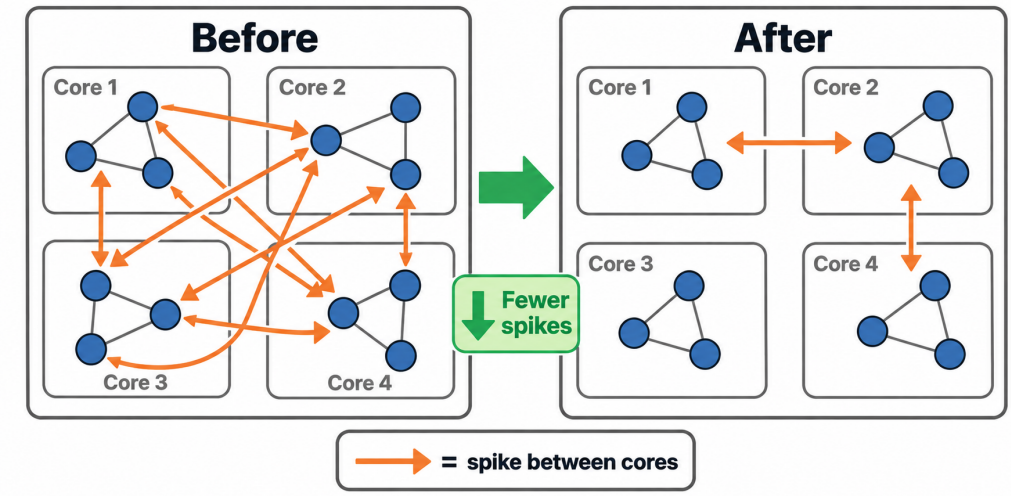


Fig. 3. The same neurons on four cores before and after hypergraph-aware mapping. Fewer spikes have to cross between cores.

TABLE V
THE SIX SEIZURE MODELS AND THE ACCURACY GATE (AUC).

Model	Layers	Synapses	Val	Test	Gate
SZ-S_dense	64-8-1	520	1.000	1.000	pass
SZ-S_sparse	64-8-1	56	0.414	0.048	**fail**
SZ-M_dense	64-64-16-1	5136	1.000	1.000	pass
SZ-M_sparse	64-64-16-1	496	1.000	1.000	pass
SZ-L_dense	64-128-128-64-32-1	34848	1.000	1.000	pass
SZ-L_sparse	64-128-128-64-32-1	3488	1.000	1.000	pass

input levels did not feed any neuron. Its AUC was below 0.5 on training and test data, so its score pointed the wrong way. We think the missing inputs are the cause, but we did not test this.

An AUC of 1.0 on validation and test files that have 6 and 11 seizure windows shows that the models work. It does not show that they generalize. The raw encoded signal is also easy to separate. Single features, such as the spread of levels in a window, already give an AUC of 0.97 to 0.99. This is the likely reason that every working model reaches a near-perfect score. It also means that accuracy cannot tell our models apart, so we compare them on mapping cost.

B. Nine Motor-Imagery Models

We wanted a second task with a larger, deeper network, so that the mapping question could be tested away from the small seizure models. We trained motor-imagery models in three sizes, MI-S, MI-M and MI-L, each in three modes: dense, pre-sparse, and post-sparse. Fig. 4 shows the model and the modes, and Table VI gives the sizes.

The model starts with a depthwise Conv1d (kernel 51, stride 5, padding 25, 22 groups) and BatchNorm. This is the front end (FE). Its output is an analog current into the LIF "conv units". Then, for each hidden layer, we use a masked fully connected layer, LayerNorm (LN), and LIF. The readout is a non-spiking leaky integrator (beta 0.9) averaged over $T =$

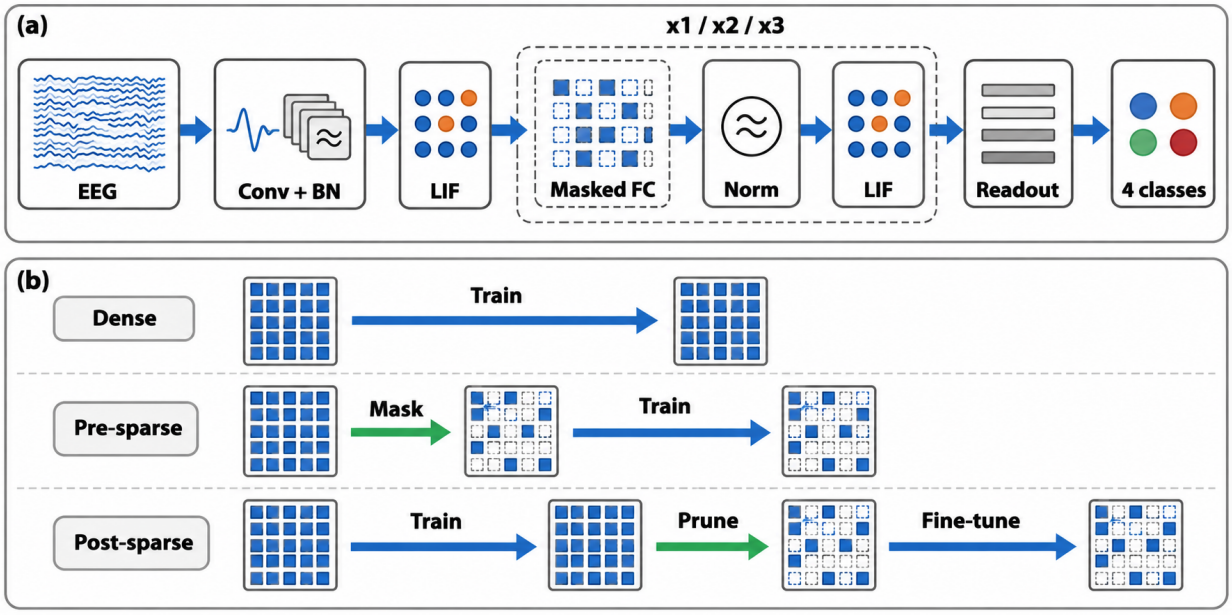


Fig. 4. (a) Motor-imagery model: depthwise conv with BatchNorm, LIF conv units, masked FC + LayerNorm + LIF blocks (1, 2 or 3 for S, M, L), non-spiking readout. (b) The three training modes: dense, pre-sparse (mask then train), post-sparse (train, prune half, fine-tune).

TABLE VI
THE THREE MOTOR-IMAGERY MODEL SIZES. FC = FULLY CONNECTED.

Size	S	M	L
Layers	22-44-64-4	22-88-128-64-4	22-176-256-128-64-4
Parameters	5,752	25,324	97,748
Hypergraph nodes	112	284	628
FC connections, dense to half	3,072 to 1,664	19,712 to 9,984	86,272 to 43,264
Masked FC layers	1	2	3
Retained parameters at 50 percent	4,344	15,596	54,740
Cores (32 per core)	4 (2×2)	9 (4×4 grid)	20 (8×8)

201 steps. The LIF neuron has a learnable per-neuron leak that starts near 0.9, threshold 1.0, reset by subtraction, and a fast-sigmoid surrogate gradient with slope 25. Conv, linear and norm operations use bf16, and membranes stay in float32. Only the FC layers between the conv units and the readout are sparse. The conv and readout stay dense.

We trained for 150 epochs with AdamW (learning rate 3×10^{-3}, weight decay 10^{-2}), 3 warm-up epochs and then a cosine schedule, dropout 0.1, label smoothing 0.1, random-roll and noise augmentation, gradient clipping at 1.0, and batch size 128. With 32 steps per epoch this is 4,800 updates. For the pre-sparse mode, a random fixed fan-in mask keeps half of the inbound connections of every neuron, and it is applied before training. For the post-sparse mode, we take the best-validation dense checkpoint, keep the top-$|W|$ half per output neuron, and fine-tune for 50 epochs (learning rate 10^{-3}). This is 1,600 more updates than the other modes. Both modes use the same mask mechanism, and pruned weights get exactly zero gradient.

Our accuracy gate for this task was 50 percent on validation. All nine models passed it, well above chance (25 percent) and above the LDA reference (44.9 percent). Published motor-imagery SNN papers report 82 to 84 percent on IV-2a with per-subject training and session-wise splits. Our 58 to 64 percent comes from pooled subjects, mixed sessions, one split, no test set, and a recipe that was not tuned for accuracy. The numbers are not comparable.

C. Mapping Setup

We used 32 neurons per core. This is a deliberately small core, chosen so that a 417-neuron network has something to split. A real Loihi-style core holds 1024. The five valid seizure models need 3, 5, 5, 14, and 14 cores. Cores sit on a square grid (4×4 for 14 cores). Firing rates were measured on the validation windows. We also built hypergraphs from the training windows, and the per-neuron rates of the two sets agree with a correlation of 0.990 to 0.998, so the choice of split is unlikely to change the findings. The hypergraphs have 73, 145, 145, 417, and 417 nodes. We checked that input rates sum to 4.000 and that synapse counts match the models. We also checked that each trained model reloads with the same output as saved (maximum difference zero).

The motor-imagery models use the same setup. We used 32 neurons per core, the same Loihi-based constants, and 1,500

TABLE VII
NAIVE VS HYPERGRAPH-AWARE PIPELINE FOR THE SEIZURE MODELS (SEED 42). GAINS ARE IN PERCENT.

Model	Cores	ELP naive	ELP hyper.	ELP	Energy	Lat.
SZ-S_dense	3	501.6	344.4	31.3	4.6	28.0
SZ-M_dense	5	6908.5	2223.5	67.8	38.2	48.0
SZ-M_sparse	5	4557.7	1471.3	67.7	38.3	47.7
SZ-L_dense	14	43583.6	34332.5	21.2	11.1	11.3
SZ-L_sparse	14	33864.7	26734.0	21.1	13.5	8.7

swaps, with 5 placement seeds for B and D. The four variants A to D are the same as before. Rates were measured on the validation windows with the best-validation checkpoint. The hypergraph synapse counts equal the model counts for every model. The depthwise conv is not modelled as synapses. The core counts are in Table VI.

VII. RESULTS

A. Hypergraph-Aware vs Naive Pipeline (Seizure Models)

Table VII compares the two full pipelines for a single run. The hypergraph-aware pipeline is better on energy, latency, and ELP for all five seizure models (Fig. 5 shows the same numbers as bars). Energy is the most careful number, with gains from 4.6 to 38 percent. ELP gains look bigger because ELP multiplies two reductions.

A single run is noisy because the swap step is random, and single runs move by about 3 to 20 percent. Over five seeds the mean ELP gains of the full pipeline are 31.3, 73.1, 70.0, 23.4, and 27.1 percent for the five models.

B. Where the Gain Comes From (Seizure Models)

Table VIII shows the ablation. Look first at B against A. Giving the naive partition the same Hilbert placement and refinement already brings most of the gain: 47.7, 76.3, 72.1, 20.1, and 18.1 percent. In other words, placement does most of the work. For the small and medium models, placement accounts for 100 percent or more of the total gain. For the two large models the share is 86 percent (SZ-L_dense) and 67 percent (SZ-L_sparse).

The fair test of partitioning is D against B. Here the answer depends on the model. On the small and medium models, overlap partitioning is neutral or worse. On SZ-L_dense it adds 4.1 percent, which only just passes our rough screen (the margin is larger than the sum of the two standard deviations). On SZ-L_sparse it adds 11.0 percent, and this one clearly passes. For the seizure models, we think the reason is wiring. In a dense layer, every neuron talks to every neuron in the next layer, so any grouping looks the same to a partitioner. In the dense seizure models the synaptic reuse is identical for both pipelines (29.42 for SZ-L_dense). Sparse wiring leaves different sets of shared inputs, so grouping can find something. This explanation fits the seizure numbers, but we did not test it directly. It does not carry over to the motor-imagery models, where partitioning helped on dense networks too (Section VII-D).

Two more points are worth stating. First, C against A is mixed. On the large models, overlap partitioning with row-by-row placement is worse than naive (by 4.6 and 8.4 percent). We believe this is because the overlap method numbers its cores in the order it seeds them, which scatters communicating cores in a row layout, while naive core IDs follow the layers. We did not test this. It does suggest that the overlap partitioner needs good placement to pay off. Second, the placement step also uses the hypergraph cost (weighted traffic drives the swaps). So "mostly placement" does not mean the hypergraph did nothing. It means the hypergraph cost model pays off mainly at the placement step in these networks.

For SZ-S_dense, the head-to-head table shows a 31.3 percent gain, but the ablation shows that naive partitioning with refined placement is better still. Both statements are true. The pipeline beats naive, but placement is the reason, not partitioning.

C. Does It Repeat? (Seizure Models)

An earlier run used different trained models (four EEG models, one with a random 90 percent sparse mask). There the placement step also dominated, and overlap partitioning clearly helped only on the deep random-sparse model, by 13.7 percent. In the present run the same kind of model gives 11.0 percent. These are two independent sets of trained models. What does not repeat is the sign for the dense deep model (within noise both times) and for the small model (0 percent then, -31 percent now). We note that the overlap partition of a 3-core network depends on exact firing rates, so it can change when the model is retrained.

D. Motor-Imagery Results

1) Accuracy: We first check that the nine models work, because a mapping study needs trained networks. Table IX gives the validation accuracy. We use the mean of the last 10 epochs as the main number, because the best-validation epoch is a noisy maximum (the M post value is a fine-tune epoch). Accuracy rises with size, from about 56 to 58 percent for MI-S to about 62 percent for MI-L.

The M and L models overfit: their training accuracy is 77 to 83 percent (M) and 88.5 to 92.6 percent (L), while validation stays near 61 to 62 percent. We did not try to fix this, because accuracy is only a gate here.

We then asked whether pre-sparse or post-sparse training is better. In last-10 points, post minus dense is -2.11 (S), -0.46 (M), and $+0.14$ (L). Pre minus dense is -1.19, -0.67, and -0.60. Pre minus post is $+0.93$, -0.20, and -0.73. The signs flip across sizes, the post mode got extra updates, and we have one seed. So pre versus post is not resolved. One-shot pruning before fine-tuning is a shock: S falls from 58.63 to 43.39, M from 62.87 to 57.38, and L from 63.65 to 59.40.

2) Hypergraphs: To map the models we need their firing rates and hyperedges, so we measured them next (Fig. 7). Rates rise with depth: about 0.04 to 0.07 at the conv units, and up to about 0.21 to 0.23 in the last hidden layer. Half-sparse models carry about 40 to 50 percent fewer synaptic events per

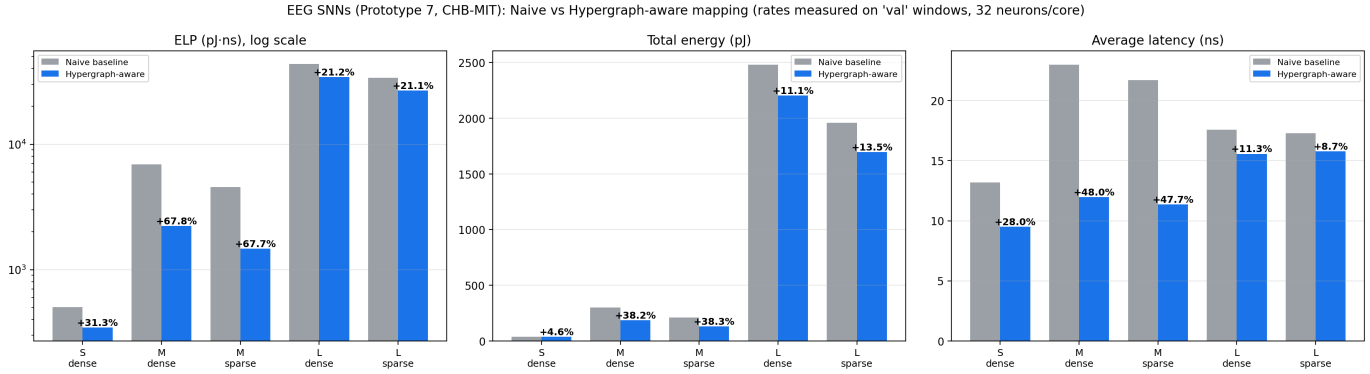


Fig. 5. Naive and hypergraph-aware mapping for the five valid seizure models (single run, seed 42). Left: energy-latency product (log scale). Middle: total energy. Right: average latency. The percentage on each blue bar is the gain of the hypergraph-aware pipeline. Seizure models; plot labels drop the SZ prefix.

TABLE VIII

ABLATION OF THE SEIZURE MODELS OVER FIVE PLACEMENT SEEDS. ELP VALUES ARE MEANS (LOWER IS BETTER); GAINS ARE IN PERCENT (POSITIVE IS BETTER). A: NAIVE + ROW PLACEMENT; B: NAIVE + REFINED PLACEMENT; C: OVERLAP + ROW PLACEMENT; D: OVERLAP + REFINED PLACEMENT.

Model	ELP A	ELP B	ELP C	ELP D	D vs A (full)	B vs A (placement)	D vs B (partitioning)
SZ-S_dense	502	262	344	344	+31.3	+47.7	−31.3
SZ-M_dense	6909	1637	2483	1860	+73.1	+76.3	−13.6
SZ-M_sparse	4558	1269	2060	1367	+70.0	+72.1	−7.7
SZ-L_dense	43584	34821	45576	33404	+23.4	+20.1	+4.1
SZ-L_sparse	33865	27751	36717	24689	+27.1	+18.1	+11.0

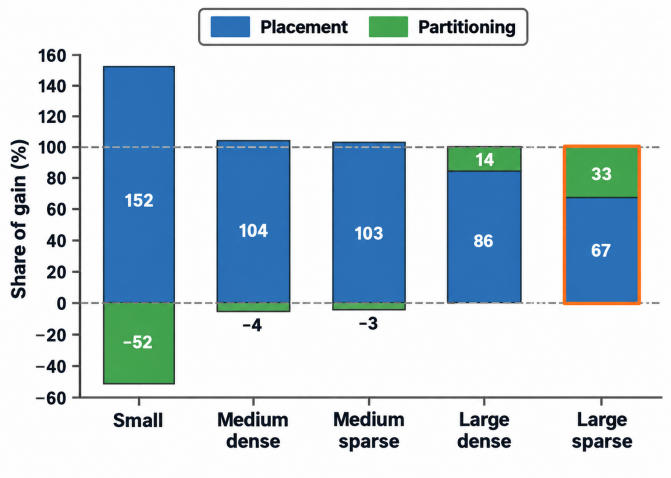


Fig. 6. Share of the total ELP reduction that comes from placement and from partitioning, for each model. Seizure models; plot labels drop the SZ prefix.

step. The values are 223 to 135 and 129 for S, 1,347 to 813 and 675 for M, and 5,714 to 3,094 and 2,924 for L, where the two numbers after "to" are the two half-sparse modes. The synapses fall from 3,072 to 1,664 (S), 19,712 to 9,984 (M), and 86,272 to 43,264 (L). Plot labels such as "S_dense_s42" mean MI-S dense, seed 42.

3) Mapping ablation: We then ran the four mapping variants A to D on all nine models to see where the gain comes from (Table X, Fig. 8). The full pipeline D beats the naive pipeline A in 9 of 9 models. The gain is +28 to +31 percent

TABLE IX

VALIDATION ACCURACY (PERCENT) OF THE NINE MOTOR-IMAGERY MODELS: BEST-VALIDATION EPOCH, AND MEAN OF THE LAST 10 EPOCHS ± STD.

Model	Mode	Best-val	Last-10 mean
MI-S_dense	dense	58.63	58.05 ± 0.28
MI-S_pre	pre	58.34	56.87 ± 0.43
MI-S_post	post	58.15	55.94 ± 0.38
MI-M_dense	dense	62.87	61.60 ± 0.46
MI-M_pre	pre	61.62	60.94 ± 0.31
MI-M_post	post	63.74	61.14 ± 0.47
MI-L_dense	dense	63.65	62.46 ± 0.49
MI-L_pre	pre	64.22	61.86 ± 0.32
MI-L_post	post	63.55	62.59 ± 0.63

for S, +44 to +46 for M, and +58 to +61 for L, so it grows with size.

Placement. B against A is exactly 0 for the S models. They have only 4 cores, and the swap search reaches the exhaustive optimum, so there is nothing more to gain. For M it is +33 to +35 and for L +50 to +54. Part of this is layout shape and not only core order, because a Hilbert layout is more compact than row-by-row.

Partitioning. D against B gives the effect of overlap partitioning on top of the same placement. It is +28 to +31 percent for S, +17 to +18 for M, and +15 to +17 for L. So on these networks partitioning pays off even on dense models. This is different from the seizure models, and we did not test why.

C against A. Overlap partitioning alone gives mixed results. It is worse than naive on the S models (−29 to −51 percent), and better on M (+21 to +31) and L (+6 to +23). We tested two

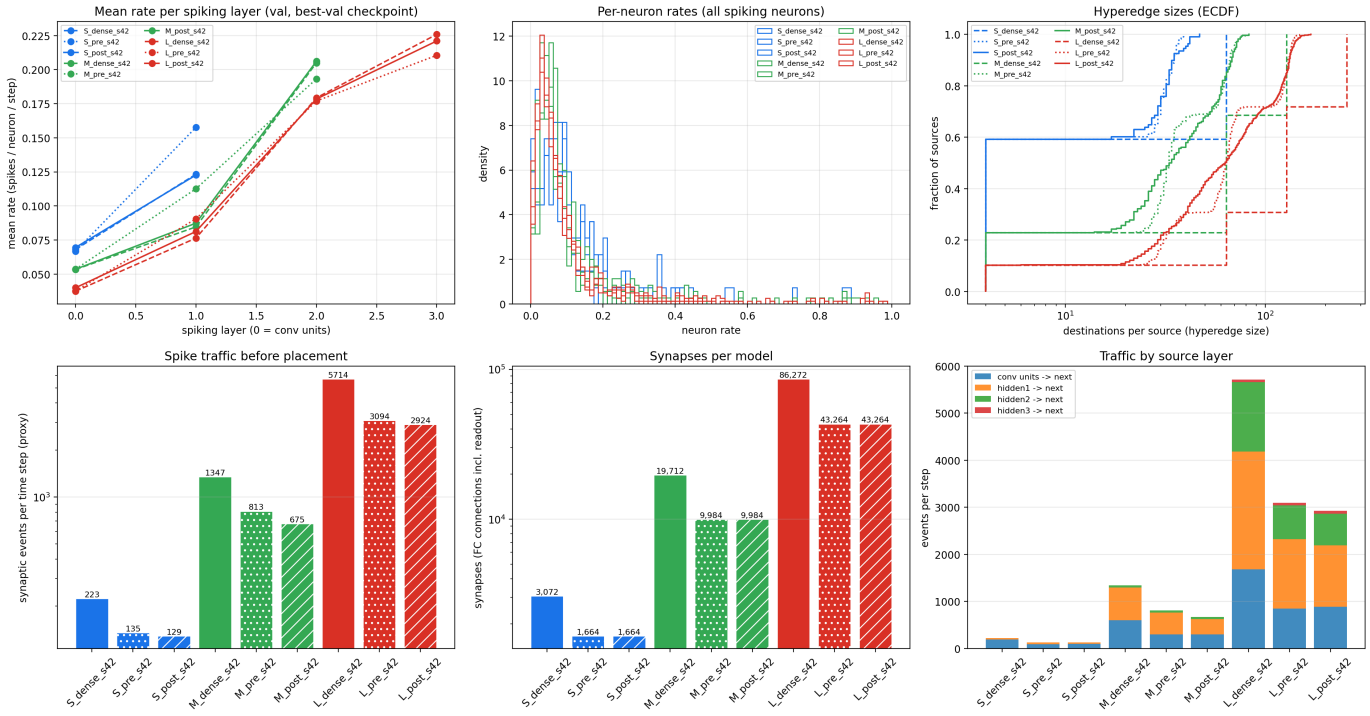


Fig. 7. Hypergraph extraction for the motor-imagery models. Mean firing rate per spiking layer; per-neuron rate histogram; hyperedge-size ECDF; spike traffic per time step before placement (log); synapses per model (log); traffic by source layer.

TABLE X
ELP OF THE NINE MOTOR-IMAGERY MODELS (pJ · ns, LOWER IS BETTER). B AND D ARE 5-SEED MEANS (STD 2 PERCENT OR LESS). GAINS ARE IN PERCENT (POSITIVE IS BETTER).

Model	A	B	C	D
MI-S_dense	1298.4	1298.4	1801.5	909.7
MI-S_pre	1438.0	1438.0	2177.2	1043.1
MI-S_post	1315.0	1315.0	1699.0	908.3
MI-M_dense	14749.5	9929.5	11578.4	8232.2
MI-M_pre	17538.6	11444.2	12191.8	9426.4
MI-M_post	15144.1	9994.0	11338.4	8199.9
MI-L_dense	133392.8	61743.6	102366.2	52608.0
MI-L_pre	137271.6	68423.6	129695.5	57159.3
MI-L_post	138486.7	65417.7	106663.6	55753.0
Gains (S / M / L)				
D vs A: +28 to +31 / +44 to +46 / +58 to +61				
D vs B: +28 to +31 / +17 to +18 / +15 to +17				
B vs A: 0 / +33 to +35 / +50 to +54				
C vs A: −29 to −51 / +21 to +31 / +6 to +23				

causes separately on the S models. One was that conv units have no inbound overlap. The other was that overlap cores are numbered in firing-rate order, so linear placement scatters them. Both closed 0 percent of the S gap (Fig. 8c). On M and L, renumbering made C worse. In S, the extra cost of C comes from hidden-layer sources (65.4 percent of A's total, against 39.4 percent for A). The cause is open.

Sparsity. Sparsity at 50 percent gives no analytical gain. Pre-sparse D is 8.7 to 14.7 percent worse than dense D, and post-sparse D is about equal. The data support this reason: the cost counts one message per (source neuron, destination core) pair, and about 99 percent of those pairs survive 50 percent fan-in sparsity, although synaptic events per step fall by 40 to 50 percent.

4) Accuracy versus cost: Finally, we checked that the ELP gain costs nothing in accuracy (Fig. 9). Mapping only moves neurons, so the accuracy of every model is the same under every mapping. The gain is therefore free in accuracy terms. ELP should be compared only inside one panel, because the three sizes have different core counts.

E. What Repeats Across the Two Tasks

We compare the two tasks to see which findings are general. Traffic-aware placement, used in the full pipeline, lowers ELP on every model of both tasks. On the MI-S models the placement-only variant B equals A, because 4 cores leave nothing to improve. Mapping never changes accuracy, so accuracy stays a gate.

Partitioning differs. For the seizure models, partitioning adds a clear gain only on the large sparse network (about 11 percent). For the motor-imagery models, overlap partitioning adds 15 to 31 percent beyond placement, even on dense models. So "dense networks give the partitioner nothing to do" is true for the seizure models only. We did not test the reason. One untested hypothesis is that the motor-imagery networks are wider and deeper, with conv-unit sources and a wide range of hyperedge sizes (Fig. 7).

C against A has opposite signs on the two tasks for the small models. On the small seizure models, C has a lower

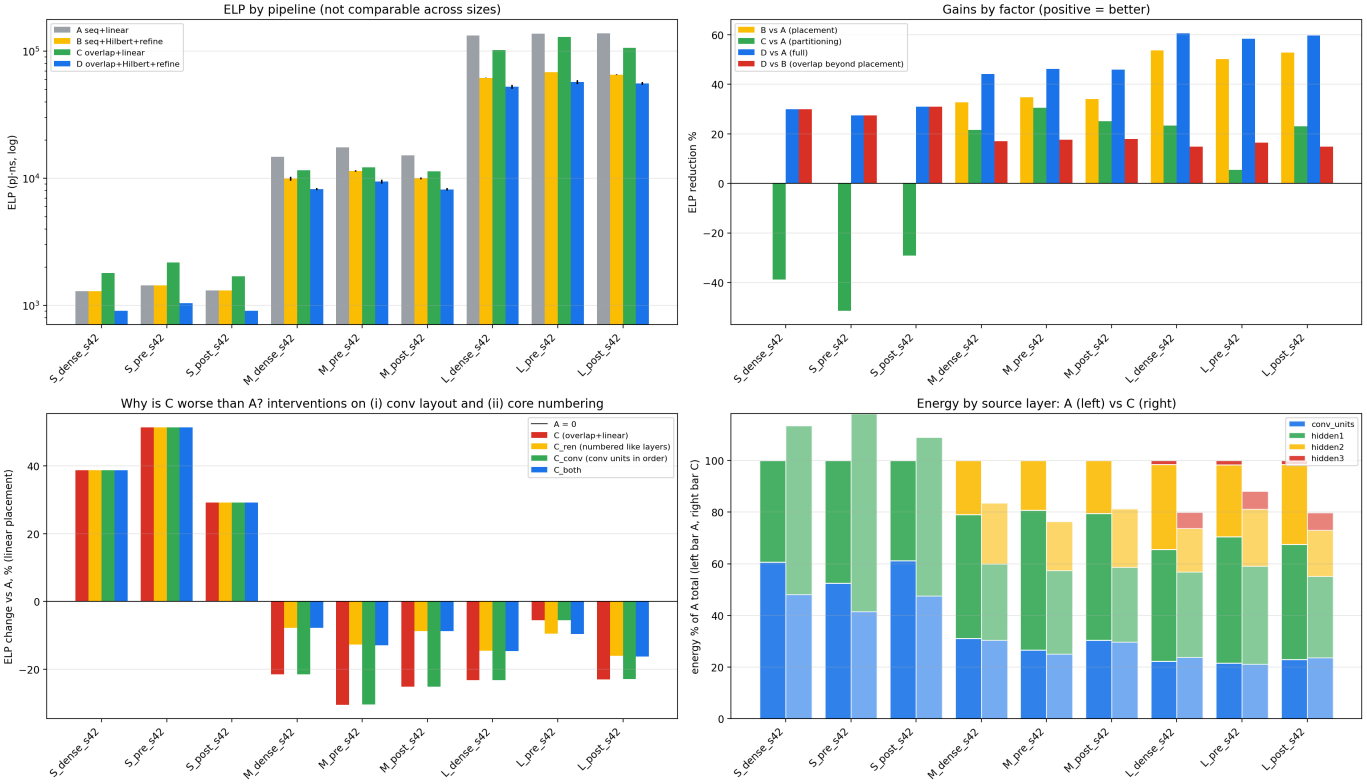


Fig. 8. Mapping ablation for the motor-imagery models. (a) ELP of A, B, C, D (log). (b) Gain by factor (positive is better): B vs A is placement, C vs A is partitioning, D vs A is the full pipeline, D vs B is overlap beyond placement. (c) Test of why C is worse than A on the S models, by changing conv layout and core numbering. Here the bars show the ELP change, so positive means worse. Neither intervention closed the gap. (d) Energy by source layer, A (left bar) vs C (right bar).

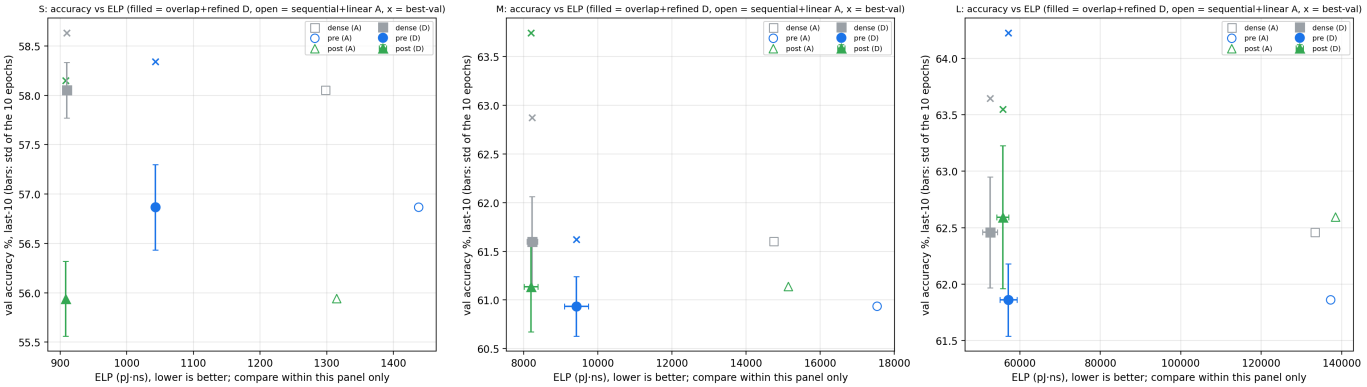


Fig. 9. Accuracy against ELP for the motor-imagery models, in three panels S, M, L. Validation accuracy is the mean of the last 10 epochs, and bars show the std. Open marks are pipeline A, filled marks are pipeline D, and x marks the best-validation epoch. Compare ELP only inside one panel.

ELP than A (Table VIII). On the MI-S models, C is worse than A (Table X). The cause is open.

Sparsity also differs. The seizure models use 90 percent sparsity, and the sparse ones fit the audited core limits. The motor-imagery models use 50 percent sparsity, and none of them fit. At 50 percent, sparsity saved no ELP and almost no memory.

VIII. Hardware Verification

The formulas above make many assumptions. They assume an empty network, average rates, and one cost per hop. We therefore checked the results at four levels, and each level removes one assumption of the one before it (Fig. 10). Level one is the formula. Level two replays real spike traces. Level three is a cycle-level simulation of the mesh with buffers. Level four is register-transfer-level (RTL) counters. Silicon is future work. All levels are simulations. They read the saved models

TABLE XI
QUANTIZATION OF THE SEIZURE MODELS. AUC IS THE SAME BEFORE AND AFTER FOR EVERY MODEL.

Model	Val AUC	Test AUC	FP32 (KB)	INT8 (KB)
SZ-S_dense	1.000	1.000	2.03	0.51
SZ-M_dense	1.000	1.000	20.06	5.02
SZ-M_sparse	1.000	1.000	1.94	0.48
SZ-L_dense	1.000	1.000	136.12	34.03
SZ-L_sparse	1.000	1.000	13.62	3.41

TABLE XII
CORE MEMORY AUDIT OF THE SEIZURE MODELS. N = NAIVE, O = OVERLAP. LIMITS: 32 NEURONS, 128 AXONS, 512 SYNAPSES PER CORE.

Model	Max synapses/core	Axon table entries	Fits (N / O)
SZ-S_dense	520 / 512	64 / 48	no / yes
SZ-M_dense	2048 / 2048	192 / 208	no / no
SZ-M_sparse	192 / 192	176 / 181	yes / yes
SZ-L_dense	4096 / 4096	1120 / 1120	no / no
SZ-L_sparse	416 / 416	1078 / 971	yes / yes

and change no trained weight.

A. Seizure Checks

The formulas above assume an empty network, 32-bit floats, and only one core limit. We therefore ran four checks without changing any trained weight. We simulate the chip and do not measure a board. A development board draws hundreds of milliwatts of background power, while the on-chip network of SZ-L_sparse uses only a few microjoules for a whole 8 s window (4.4 to 5.2 μJ in our model). A wall-power meter could not see that difference. Fig. 11 gives an overview of the three main results below.

1) 8-bit quantization: We turned the weights into signed 8-bit integers by scaling each layer's largest weight to 127. Membrane values use 16-bit integers, and the leak of 0.9 becomes a multiply by 230 followed by a right shift of 8 (230/256 $\approx$ 0.898). All five models kept an AUC of 1.000 on validation and test data, with zero loss. Weight memory dropped by 75 percent (Table XI).

2) Core memory audit: Until now we only enforced the neuron limit. We now check all three limits with scaled values of 32 neurons, 128 inbound axons, and 512 synapses per core. These follow the 1:4:16 ratio of the paper. Table XII shows the result. The axon limit was met everywhere. The synapse limit is where models fail. Both dense models with more than one layer (SZ-M_dense and SZ-L_dense) overflow by a factor of 4 to 8. Both sparse models fit. For SZ-S_dense, the naive partition needs 520 synapses in one core and overflows by 8, while the overlap partition needs exactly 512 and fits. For SZ-L_sparse, overlap partitioning cuts the total number of inter-core axon table entries from 1078 to 971 (9.9 percent less). Note that these limits are chosen by us, so the audit shows the effect of sparsity and does not prove that any specific chip would accept the models.

3) Cycle-accurate mesh replay: Our formulas use average firing rates, and 98 to 99 percent of the windows are calm EEG. A seizure is rare, but it makes many neurons fire at once, and this is when the chip is under the most stress. We built a trace-driven simulator in the style of [30], [31]. We replayed the real spike trains of the held-out file (record 18) through a 2D mesh with X-then-Y routing, one step at a time. At each of the 2048 steps, every spiking neuron sends one packet to each other core that holds a target. If more than one packet wants the same link in the same step, the extra packets wait in a buffer. In our model, each wait adds 2.1 ns and 0.45 pJ. We replayed one normal window and one seizure window.

Table XIII gives the results, and there are three things to read from it. First, the number of packets for SZ-L_dense is exactly the same in both pipelines. In a dense network, a neuron must reach every core that holds the next layer, whatever the grouping. For SZ-L_sparse, overlap partitioning removes 14,523 packets in the seizure window, and the packet count is a pure partitioning effect. Second, on the small and medium models the hypergraph-aware pipeline actually sends more packets, but over much shorter paths (average hops per packet fall from about 2.7 to 1.3 on SZ-M_sparse). This is a placement effect, and it removes most of the stalls. Third, SZ-L_sparse gains more during the seizure window (27.6 percent ELP, 16.6 percent dynamic energy, and 691,758 fewer stall cycles) than in the normal window (13.5 percent). Since this replay compares naive partitioning and placement against the full pipeline, it cannot separate the two parts, and it covers one window of each kind.

4) Verilog router simulation: Finally, we wrote Verilog for the 2D XY-router mesh of SZ-L_sparse in two versions, one with the naive mapping tables and one with the hypergraph-aware tables. We simulated the first 256 seizure clock cycles at 100 MHz in Icarus Verilog and also ran the logic through Yosys synthesis. We do not report area or timing numbers here. The hardware counters in Table XIV agree with the Python simulator on the size of the reduction.

This is a simulation of the routing and counting logic for 256 cycles, not a finished chip. What it does show is that the lookup tables and the routing counts can be written as hardware, and that the numbers match the software model.

B. Motor-Imagery Checks

For the motor-imagery models we asked four questions, Q1 to Q4, and then checked RTL counters (Table XV). First we made sure that the checks themselves are sound. Reloading the saved models reproduces all saved accuracies exactly. The saved spike traces reproduce bit for bit. BatchNorm folds exactly into the conv. Regenerated placements reproduce the saved ELP tables. The cycle-level simulator passes its validation suite: single-packet latency equals hops plus 1, packets are conserved, hop counts equal Manhattan sums, runs are deterministic, there is no back-pressure with unbounded buffers, and it agrees with trace replay on all 1,152 main runs.

1) Q1: Does the gain survive simulation?: We asked this because a formula can hide effects such as waiting in buffers. The answer is yes (Fig. 12). The D vs A gain is +27.5 to +61.7

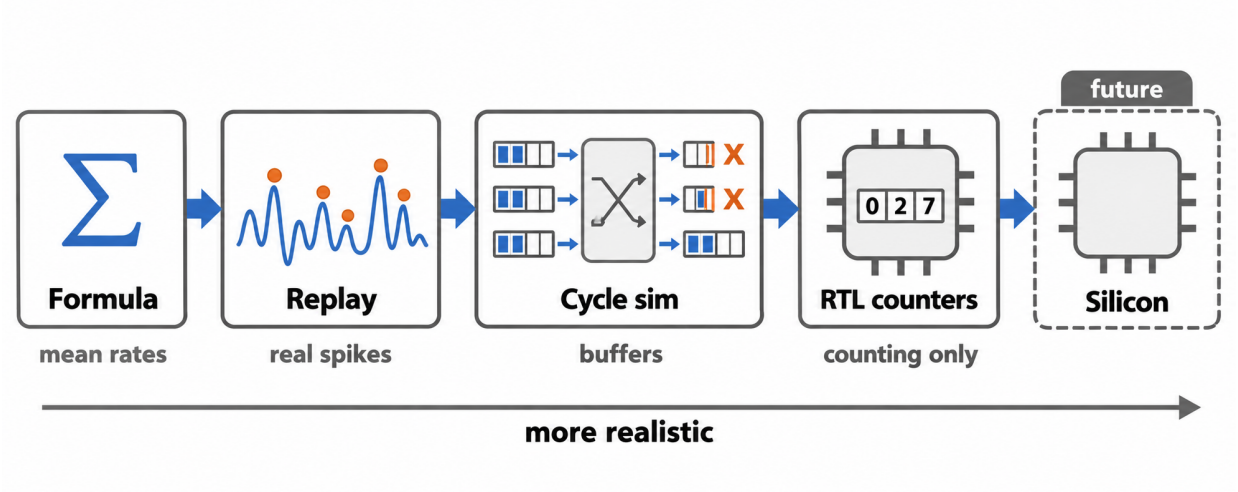


Fig. 10. The four-level verification ladder: formula, trace replay, cycle-level mesh simulation, and RTL counters, with silicon as future work. Each level removes one assumption of the one before it.

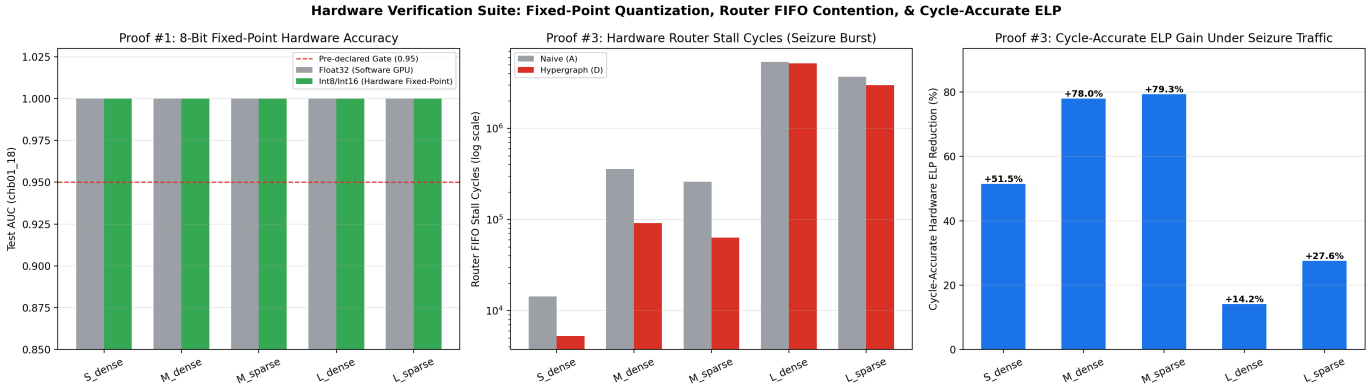


Fig. 11. Hardware verification results for the seizure models. Left: test AUC stays at 1.0 after 8-bit quantization for all five models (the dashed line is the 0.95 gate). Middle: router buffer stall cycles in the seizure window, log scale. Right: cycle-accurate ELP gain of the hypergraph-aware pipeline in the seizure window.

TABLE XIII
CYCLE-ACCURATE REPLAY OF ONE SEIZURE WINDOW OF RECORD 18 (N = NAIVE, H = HYPERGRAPH-AWARE). "STALLS CUT" IS THE DROP IN BUFFER STALL CYCLES. ELP GAINS ARE SHOWN FOR THE NORMAL WINDOW AND THE SEIZURE WINDOW.

Model	Packets N	Packets H	Hops N	Hops H	Stalls cut (%)	ELP gain normal (%)	ELP gain seizure (%)
SZ-S_dense	8,192	8,965	12,288	8,965	63.0	+27.4	+51.5
SZ-M_dense	35,045	43,105	97,476	59,357	74.5	+82.3	+78.0
SZ-M_sparse	31,047	36,590	84,712	46,485	75.6	+75.6	+79.3
SZ-L_dense	342,792	342,792	717,077	622,819	3.3	+15.0	+14.2
SZ-L_sparse	280,926	266,403	597,309	494,383	18.8	+13.5	+27.6

TABLE XIV
VERILOG SIMULATION OF THE SZ-L_SPARSE ROUTER MESH, FIRST 256 SEIZURE CYCLES AT 100 MHZ.

Counter	Naive	Hypergraph	Change
Packets	34,407	32,616	−5.2%
Mesh hops	73,724	60,220	−18.3%
Router energy (pJ)	441,856	368,591	−16.6%

percent in the formula and +27.6 to +61.8 percent in trace replay. In the cycle-level model it is +31.7 to +59.9 percent in ELP, +16.3 to +33.9 percent in network time, and +16.8 to +31.0 percent in packet latency. Pipeline D confirms the analytical gain within 5 points in 9 of 9 models, and it has the lowest cycle-level ELP in 9 of 9. The replay used 32 real spike windows (201 steps). With first-in first-out (FIFO) buffer depths of 2, 4, 8 and unbounded (on 4 windows), the D vs A ELP gain stays between +32 and +61 percent.

Energy uses the same hop formula at every level, so the

TABLE XV
SUMMARY OF THE MOTOR-IMAGERY HARDWARE CHECKS. ALL CHECKS ARE SIMULATIONS.

Check	Question	Result
Q1	Does the gain survive simulation?	Yes. D vs A: analytical +27.5 to +61.7, trace-driven +27.6 to +61.8, cycle-level ELP +31.7 to +59.9, cycle-level network time +16.3 to +33.9 percent. D has the lowest cycle-level ELP in 9 of 9.
Q2	What if extra traffic that the formula ignores is added?	The gain shrinks with LN and FE traffic. With a single FE input port it can vanish: -23.1 to $+8.8$ percent at the best corner.
Q3	Does int8 keep the accuracy?	The declared gate failed (1 of 9 models with LN a, 0 of 9 with LN b). int8 minus float32 is small: -1.4 to $+1.3$ points. The gate sits at the numerics floor.
Q4	Do the mappings fit the memory and capacity limits?	No. 18 of 18 partitions violate the synapse limit, and 9 of 18 violate the axon limit. At 50 percent sparsity, index storage leaves a saving of only 0.5 to 5.8 percent.
RTL	Do hardware counters match the software?	88 of 88 runs equal the Python reference and the trace-replay counts. This checks accounting logic only.

TABLE XVI
D VS A CYCLE-LEVEL ELP GAIN (PERCENT) FOR THE SIX TRAFFIC SCENARIOS, AS RANGES OVER THE NINE MOTOR-IMAGERY MODELS.

Scenario	Gain range
Spike only	+31.7 to +59.9
Plus LN	+29.3 to +59.2
Plus FE packed (one packet per hosting core)	+20.3 to +57.9
Plus FE per unit, 4 corner ports	+12.1 to +36.0
Plus FE per unit, best single corner	-23.1 to $+8.8$
Plus FE per unit, corner (0,0)	-70.2 to -3.9

ELP agreement is partly an energy agreement. Latency and network time are the independent evidence. Placement alone (B) is weaker in simulation: its network time is worse than A in 3 of 9 models, the L ones. Overlap partitioning alone (C) helps M and L, but the S models contradict it.

2) Q2: Extra traffic the formula ignores: The formula counts spikes only, so we added the other traffic of the model. The conv layer feeds analog current to the first spiking layer at every step (the FE traffic). Each LN needs a cross-core mean and variance at every step (the LN traffic). We added these in steps (Table XVI, Fig. 13). The gain shrinks as traffic is added. Adding LN costs little. With FE traffic packed into one packet per hosting core, the gain is still +20.3 to +57.9 percent. With four corner ports it is +12.1 to +36.0 percent. With a single port it can vanish, and for the worst corner it is negative.

The single-port cases are limited by serialisation: the cycles per step are about equal to the number of conv units, whatever the mapping. They are bounds, not design predictions. The lesson is that the front end must be designed together with the mapping.

3) Q3: Does int8 keep the accuracy?: We asked this because quantisation is the usual step before a chip. We declared a gate in advance: a paired accuracy drop of at most 1.0 point and a prediction agreement of at least 95 percent. We tested two LN variants. In LN a, the LayerNorm stays float on quantised weights. In LN b, the LayerNorm is fixed-point. The gate failed as declared: 1 of 9 models passed with LN a, and 0 of 9 with LN b.

But the int8 change is small. Int8 minus float32 is -1.4 to $+1.3$ points for LN a, and -1.3 to $+0.8$ for LN b. McNemar $p < 0.05$ in only 2 of 18 comparisons. The gate was partly unattainable. Bf16 (the training precision) and float32 of the same checkpoint agree on only 93.2 to 96.5 percent of predictions and differ by -0.2 to $+1.4$ points, so 3 of 9 models have a numerics floor below 95 percent. The lower end of the confidence interval is below -1.0 point in 16 of 18 cases, so we cannot exclude a drop larger than 1 point. Fixed-point LayerNorm costs nothing extra (mean change -0.42 against -0.44). We saw no saturation events in the main configuration. Four-bit weights collapse (mean change -9.96), so they are not viable without quantisation-aware training (Fig. 14).

4) Q4: Memory and capacity: We asked whether the mappings fit a core and how much memory sparsity really saves. For memory we compared three storage formats at int8 (Fig. 15): weights only, weights plus a compressed-sparse-row (CSR) index, and a crossbar that stores the zeros. The sizes in KiB (weights only, with CSR index, crossbar) are 5.9, 5.9, 5.9 for MI-S dense; 4.5, 5.6, 5.9 for MI-S pre; 16.0, 24.3, 25.5 for MI-M pre; and 55.1, 96.6, 97.1 for MI-L pre (MI-L dense is 97.1). Weights-only savings are 23 to 43 percent. Once index storage is counted, the saving is only 0.5 to 5.8 percent.

For capacity, the neuron limit C_{npc} of 32 is enforced, and the limits C_{apc} of 128 and C_{spc} of 512 are only reported (Fig. 16). All 18 partitions violate C_{spc}, with the worst core 16 times over, and 9 of 18 violate C_{apc}. Example cores over the synapse limit (sequential / overlap) are 3/2 of 4 for MI-S dense, 7/6 of 9 for MI-M dense, and 15/14 of 20 for MI-L. The sequential MI-M and the MI-L mappings also violate C_{apc}. Overlap cuts the axon violations (M from 2 to 0, L from 14 to 12) but not the synapse violations. By synapse count alone the models would need 4 to 6 cores (S), 20 to 39 (M), and 85 to 169 (L), against 4, 9, and 20 used. The violation is structural: a core that holds a full 32-neuron slice already exceeds C_{spc} when the fan-in is above 16, whatever the placement. So the motor-imagery mappings are communication-cost estimates for mappings that do not fit the assumed chip.

5) RTL counters: At the fourth level, 88 of 88 RTL runs equal the Python reference and the trace-replay counts (pipelines A and D, stress and class windows, 201 ticks). All 54 self-tests pass, and synthesis with Yosys gives 1,317 to 1,621 generic cells and 284 flops. This checks the accounting logic only. It is not a router, and it shows no congestion.

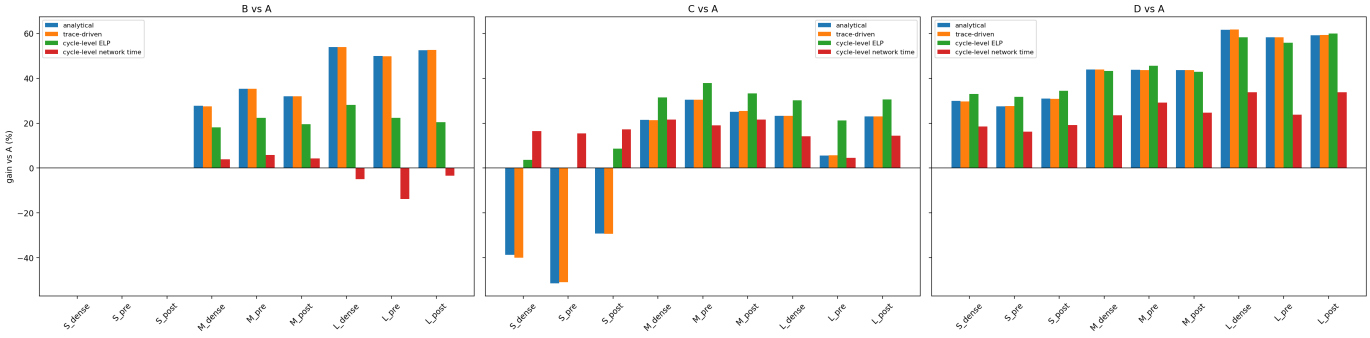


Fig. 12. Gains by evidence level for the motor-imagery models: B vs A, C vs A and D vs A. Bars show the analytical, trace-driven, cycle-level ELP, and cycle-level network-time gains. B vs A has no bars for the S models because B equals A there (4 cores, gain exactly 0). The network-time gain of B is negative for the three L models.

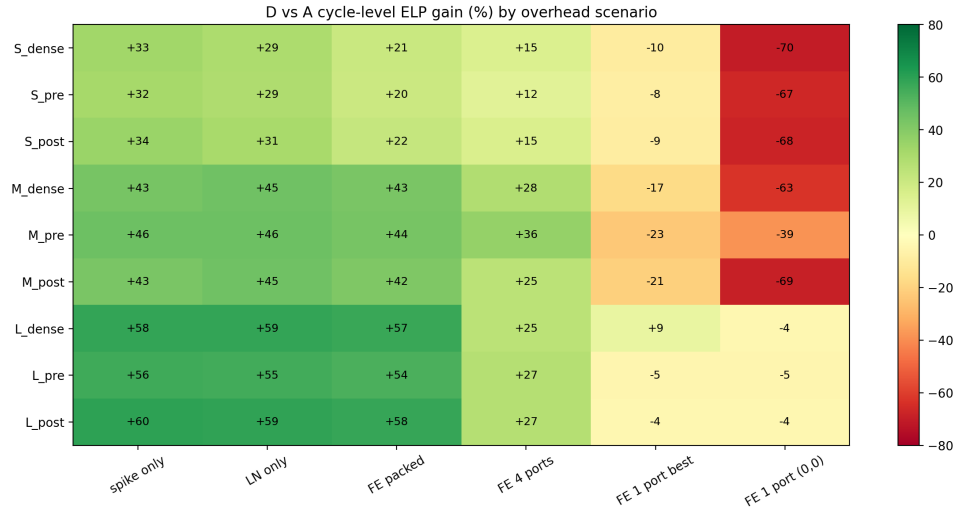


Fig. 13. Cycle-level ELP gain of D vs A (percent) for 9 models and 6 traffic scenarios: spike only, LN only, FE packed, FE 4 ports, FE 1 port best, FE 1 port at corner (0,0). FE = front end (conv layer feeding analog current to the first spiking layer), LN = LayerNorm (cross-core mean and variance). The two single-port columns are serialisation-limited bounds, not design predictions.

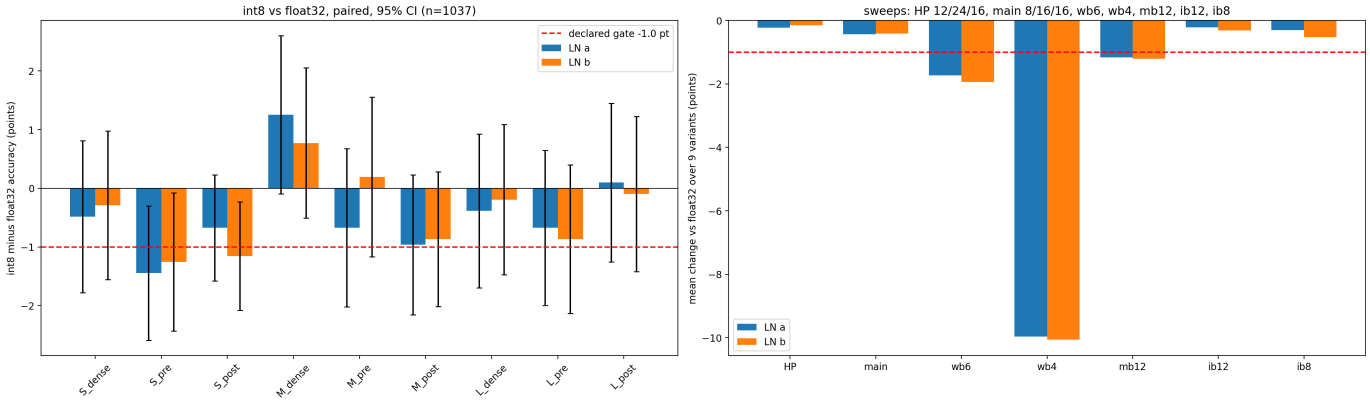


Fig. 14. Quantisation of the motor-imagery models. Left: int8 minus float32 accuracy, paired, 95 percent CI, $n = 1{,}037$, for two LayerNorm variants (LN a: float LayerNorm on quantised weights; LN b: fixed-point LayerNorm). The dashed line is the declared gate of -1.0 point. Right: mean change over 9 variants for bit-width sweeps: HP = 12-bit weights, main = int8 (8/16/16), wb6 = 6-bit weights, wb4 = 4-bit weights, mb12 = 12-bit membrane, ib12 and ib8 = 12-bit and 8-bit input. Mean changes: HP -0.23, main -0.44, wb6 -1.74, wb4 -9.96, mb12 -1.17, ib12 -0.21, ib8 -0.30.

IX. What We Learned

This section is the main point of the paper. Each result below changed what we did next. Table XVII lists the full chain, and Fig. 17 shows it as a flow. The paragraphs after them explain the reasoning.

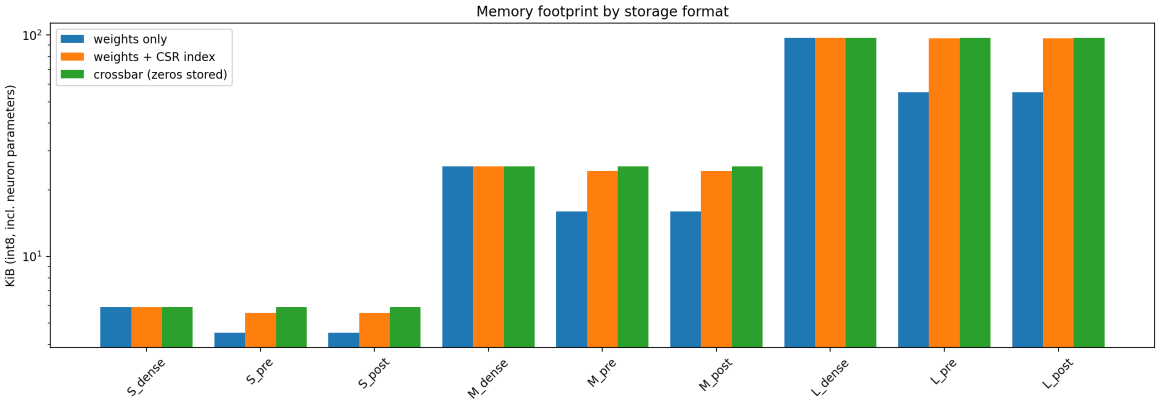


Fig. 15. Memory footprint (KiB, int8, log axis) of the motor-imagery models by storage format: weights only, weights plus CSR index, and crossbar (zeros stored).

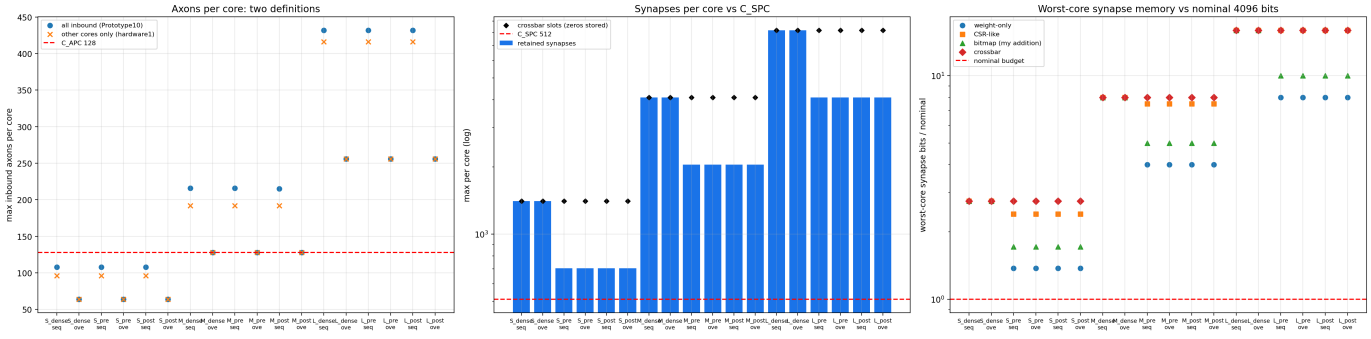


Fig. 16. Capacity audit of the motor-imagery models. Left: inbound axons per core under two definitions (all inbound sources, or inter-core only) against the limit of 128. Middle: maximum synapses per core against the limit of 512. Right: worst-core synapse memory as a multiple of a nominal 4096-bit budget for four storage formats (weight-only, CSR-like, bitmap, crossbar). seq = sequential partition, ove = overlap partition.

A. Lessons

1. More EEG channels did not mean more information. Grouping many channels into hyperedges gave a lower sensitivity and slower training than four fixed channels. Our guess is that the extra channels added noise, and we did not test this. We also felt it was wrong to keep tuning the threshold until the result looked good. We dropped the idea instead. This is a result on one patient and one split. It rules out "clearly better", but not "might help somewhere".

2. A wearable chip and a mapping study pull in opposite directions. A wearable wants the smallest possible network. Mapping only pays off when there is a network large enough to split. Once we saw this, we stopped designing for a wearable and asked a different question: for which sizes and wiring does hypergraph mapping help? Mapping also cannot change accuracy. It only changes where neurons live. So accuracy is a check, not a goal.

3. Dense wiring does not always leave the partitioner nothing to do. For the seizure models, a dense layer gave the partitioner nothing: we saw the null result on the 73-neuron model, identical synaptic reuse on `SZ-L_dense`, and identical packet counts on `SZ-L_dense` in the mesh replay. For the motor-imagery models, overlap partitioning added 15 to 31 percent beyond placement even on dense networks. We did not test why. One untested hypothesis is that these networks are wider and deeper, with conv-unit sources and a wide range of hyperedge sizes.

4. A comparison is only as good as its baseline. Our first big wins came from baselines that never put inputs next to their targets. After we fixed the baseline and used one yardstick for all methods, the wins shrank or disappeared. We now treat any large early win as a reason to check the baseline.

5. Fix the model before judging the mapping. When a network scores 0.83 or 0.70 AUC, its firing rates say little about a seizure detector. So we set an accuracy gate in advance and used a separate test file. A model that fails the gate (`SZ-S_sparse`) is out. We do not rescue it.

6. A broken tool is not a failed idea. The crashed Mt-KaHyPar call and the silent fallback taught us two things. A silent fallback hides what you are really testing. And a result from a tool that did not run is not evidence for or against the method. After that we allowed no hidden fallbacks.

7. Rebuild small, prove it on controlled data, then move to real data. Patching the old pipeline made it hard to tell a bug from a finding. Writing the code again from the paper, and testing it first on synthetic data where every model trains well, gave a pipeline we could trust and explain.

TABLE XVII
DECISION LOG: WHAT WE TRIED, WHAT HAPPENED, WHAT WE CONCLUDED, AND WHAT WE DID NEXT.

Step	What we tried	What happened	What we concluded	What we did next
1	Replicate the wearable 64-8-1 detector	AUC 0.998 and event sensitivity 1.0, but more false alarms and one dead fold	A working baseline exists. Training budget and post-processing matter	Used it as the base for all later work
2	Group EEG channels into hyperedges before encoding	Fixed 4 channels won (sens. 0.989). Groups needed more epochs and gave no gain	More channels did not help here, and the grouping needed a hand-tuned threshold	Dropped it. Moved the hypergraph to the hardware side
3	Map the real 73-neuron model and a synthetic 1000-node network	-5.6% or worse on the real model, about -24% messages on the synthetic one	The idea needs sparse structure. Our real model was too small and dense	Planned bigger, sparser, trained models
4	Check the wearable fit	529 weights fit in one core, and 61% of the FPGA logic is already used	Tiny network: nothing to split. Big network: too large for a wearable chip	Dropped the wearable as the goal. Asked when mapping pays off
5	First full pipeline with a "90% sparse winner"	Win came from untrained nets, weak baselines, and a planted mask	A comparison is only as good as its baseline	Withdrew the claim. Same metric for every method
6	Deeper seizure model with hypergraph code	Inconclusive: failed accuracy gate, 10 steps, crashed tool, duplicate code	The failure was in tools and design, not in the idea	Stopped patching. Rebuilt from the paper
7	Rebuild on synthetic data, then move to EEG models	ELP gains 33, 21, 34% (synthetic) and 48, 44, 20, 7% (EEG). Ablation: mostly placement	Most of the gain is placement. Two models were too weak to trust	Retrained all models with a pre-declared gate and a test file
8	Six-model seizure ladder with the gate	Five passed, `SZ-S_sparse` failed. Partitioning helps on the large sparse model (+11.0%)	The pipeline wins everywhere. Partitioning mattered only for large sparse wiring on this task	Checked hardware realism
9	Quantization, memory audit, mesh replay, Verilog (seizure models)	No loss at 8 bits. Dense models overflow. Seizure stalls cut. Verilog agrees	Sparsity decides whether a seizure model fits. Seizure traffic shows the biggest gain	Added a second task, motor imagery
10	Rebuild the motor-imagery code as one clean study	Old code did one optimizer step per epoch (batch 4,000 on 4,147 trials = 150 updates)	The recipe was far from converged	Retrained at batch 128 (4,800 updates); dense best-val rose from 54.7, 58.3, 57.3 to 58.6, 62.9, 63.7 (S, M, L)
11	Check saved pre-sparse files	Old pre-sparse files (batch 4,000) sat next to the new dense ones	Mixed recipes make a false comparison	Added a stale guard; retrained pre-sparse; old numbers discarded
12	Nine variants (3 sizes x dense, pre, post), A/B/C/D mapping	D beats A in 9 of 9 (28 to 62 percent), more on larger nets; 50 percent sparsity gave no ELP gain	Overlap partitioning pays off on these nets; sparsity at 50 percent does not	Tested the C-vs-A gap (two causes ruled out) and moved to hardware checks
13	Formula, trace replay, cycle-level mesh, RTL counters	Gain survives at all levels; a single front-end port removes it; int8 failed the declared gate (numerics floor); memory saving small; no mapping fits C_{spc}	The mapping gain is real but depends on the front end and on capacity	Listed the next experiments (Section X)

8. Ask where a gain comes from. The first headline was "hypergraph pipeline wins by 7 to 48 percent". The ablation changed this to "placement wins, and partitioning adds a clear gain only on a large sparse seizure network". The second sentence is less exciting, but it is what the data say, and it repeated on a second set of models (13.7 percent, then 11.0 percent).

9. Sparsity helped the seizure models fit, but not the cost at 50 percent. The 90 percent sparse seizure models fit the audited core limits, and the dense ones overflowed by 4 to 8 times. The motor-imagery models are sparse at only 50 percent. None of them fit, and sparsity saved no ELP and almost no memory once index storage was counted. About 99 percent of the (source neuron, destination core) pairs survive 50 percent fan-in sparsity, so the cost does not fall. Sparsity helps only when it is high enough to remove whole connections between cores and to bring the synapse count under the core limit.

10. Seizure traffic is the case that matters. Averaged over mostly calm EEG, `SZ-L_sparse` gained 13.5 percent ELP in the replay. During the seizure window it gained 27.6 percent. A mapping that is tuned on average traffic may look modest but still help most when the chip is busiest. Because we tested one window of each kind, we count this as a lead and not a proof.

11. Report the negative results. Several of our most useful findings were negative: the EEG-side hypergraph, the dense-network null result, the failed model, and the placement-versus-partitioning split. They show what to try and what not to try.

12. Count the optimizer steps. Our first motor-imagery run did 150 updates, because the batch was larger than a single step per epoch. A printout of steps per epoch would have caught it.

13. Guard against old files. Stale results that sit next to new ones give false comparisons. We added a guard and discarded the old numbers.

14. A gate can fail for the right reason. The int8 gate sat at the numerics floor between bf16 and float32 of the same checkpoint. We should report the floor next to the gate.

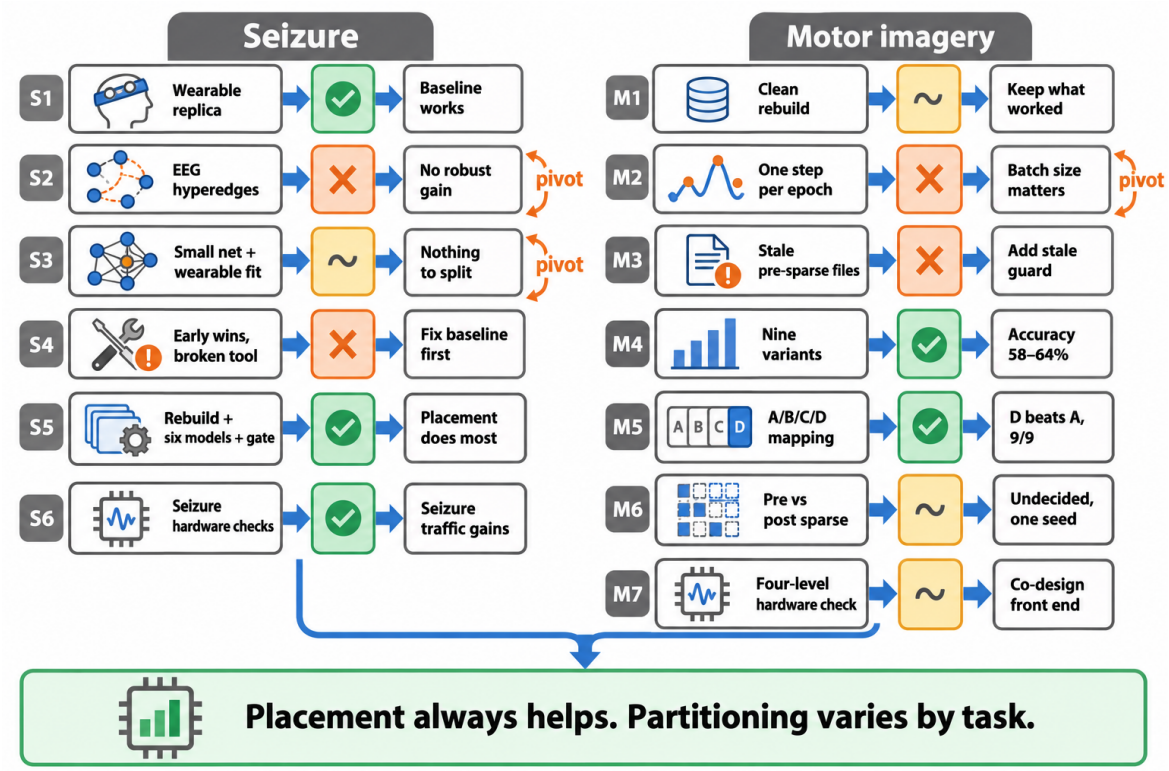


Fig. 17. How the result of each experiment guided the next one, in a seizure arc and a motor-imagery arc that end in one box. Green means the idea worked, orange means it failed, and yellow-grey means mixed. Pivot arrows mark changes of direction. Final box: "Placement always helps. Partitioning varies by task."

15. Design the front end with the mapping. A single input port removed the mapping gain. The conv traffic belongs in the mapping problem.

16. Keep evidence levels separate. The formula, the replay, the cycle-level simulation and the RTL counters are never called by each other's names. Each answers a different question.

B. The Answer We Reached

We ended with a more modest question than the one we started with: for an EEG-based SNN, when does hypergraph mapping pay off? Our data give a conditional answer. Traffic-aware placement helps on every model of both tasks. Overlap partitioning on top of it helps depending on the network. It helps clearly on the motor-imagery networks and on the large sparse seizure network, and not on the small seizure networks. Sparsity alone did not lower the cost at 50 percent. The front end must be designed together with the mapping. We think this is an honest result, and it points to what to test next.

X. Limitations

These points limit how far the results can be pushed.

- **One patient, one split.** All seizure experiments use chb01. The validation and test files each hold a single seizure event (6 and 11 positive windows). An AUC of 1.0 shows that the models work, not that they generalize.
- **No held-out test set for motor imagery.** Validation is used for model choice, firing rates and reporting. The data are pooled over subjects, the sessions are mixed, and there is one split. Training on session 0 and testing on session 1, or leave-subject-out, is next. Our accuracy is not comparable with published numbers.
- **One training seed.** The pre versus post comparison is undecided.
- **Axon and synapse limits are only audited.** They are checked after partitioning (Section VIII-A2) and are not used inside the partitioner, which is where synaptic reuse should matter most. No motor-imagery mapping fits the assumed limits, so those results are communication-cost estimates for mappings that do not fit.
- **Few baselines.** The final studies use one naive baseline. They do not compare a graph model against a hypergraph model, and Mt-KaHyPar [37] never ran in the final pipeline. There is no random placement baseline. So our claim is "hypergraph-aware pipeline versus naive pipeline", not "hypergraph model versus graph model". Our overlap partitioner is a simpler greedy version of the published algorithm.
- **Placement search is simple.** It only swaps core positions with 1500 random trials, and it never moves a core to an empty grid cell. Our verdict rule for the seizure models (margin larger than the sum of two standard deviations)

is a rough screen and not a significance test.

- **Estimates, not measurements.** Energy and latency come from a model with Loihi-small constants and small 32-neuron cores. The stall energy of 0.45 pJ and stall latency of 2.1 ns are labelled assumptions. They cannot be compared with the 4.55 nJ of the wearable FPGA.
- **Open causes.** We do not know why partitioning pays off on dense motor-imagery networks and not on dense seizure networks. We do not know why C is worse than A on the MI-S models but better on the small seizure models.
- **Rates come from mostly calm EEG.** Per-class rates (normal vs seizure) were not studied separately.
- **The replays are narrow.** The seizure replay uses one normal and one seizure window of one record, and it compares full pipelines. The motor-imagery replay uses only 32 trace windows, and the buffer-depth test uses 4.
- **The RTL models are small.** They cover counting logic for a few hundred cycles, are not a router, and show no congestion. We do not report area or timing.
- **Short training and one sparsity level.** The seizure models trained for 165 steps. The motor-imagery models use only 50 percent unstructured sparsity.
- **Int8 gate.** The declared gate was partly unattainable because of the numerics floor, and 4-bit weights need quantisation-aware training.

XI. Conclusion and Future Work

We studied how to map EEG SNNs from two tasks onto neuromorphic cores using a hypergraph model. On five seizure models that passed an accuracy gate, a hypergraph-aware pipeline lowered the estimated energy-latency product by 23 to 73 percent against a naive layer-order pipeline. An ablation showed that most of this gain comes from traffic-aware placement. Partitioning added a clear gain only on the large sparse seizure network, and this repeated across two sets of trained models. On nine motor-imagery models, the full pipeline beat the naive one in 9 of 9, by about 28 to 62 percent, and the gain grew with size. Overlap partitioning added 15 to 31 percent beyond placement, also on dense models. We do not know why this differs from the seizure models.

Mapping never changed accuracy. The gain survived trace replay and cycle-level simulation, but a single front-end input port removed it. Int8 did not pass our declared gate, which sat at the numerics floor. Sparsity at 50 percent saved almost no memory once index storage was counted, and no motor-imagery mapping fits the assumed core limits. So traffic-aware placement helps on every model of both tasks, overlap partitioning helps depending on the network, and the front end must be designed with the mapping.

The path to this result was not straight. The EEG-side hypergraph idea and the wearable target both failed, some early wins were withdrawn, and the first motor-imagery runs were far from converged. Each failure removed one wrong explanation, and this is why the final result can be trusted.

Next we plan to do the following.

- Use the axon and synapse limits inside the partitioner (capacity-aware partitioning), and add a plain-graph baseline, random placement, and a working run of an external hypergraph partitioner on the same cost function.
- Design the front end and the mapping together: put the conv traffic in the placement objective and try multi-port injection.
- Train on session 0 and test on session 1, or leave one subject out, with more than one seed, and settle pre versus post sparsity.
- Try quantisation-aware training and a fairer gate, and try structured or higher sparsity.
- Implement the published overlap algorithm closely, and improve the placement search (moves to empty cells, real annealing, more seeds, and a proper significance test).
- Replay more windows, measure per-class rates, add a second seizure patient, and build a real router in RTL before a multi-core FPGA design with measured resource use.

References

[1] P. Busia *et al.*, "Wearable epilepsy seizure detection on FPGA with spiking neural networks," *IEEE Trans. Biomed. Circuits Syst.*, vol. 19, no. 6, Dec. 2025, doi: 10.1109/TBCAS.2025.3575327.

[2] A. H. Shoeb and J. V. Guttag, "Application of machine learning to epileptic seizure detection," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 975–982.

[3] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[4] Z. Zhang, M. Xiao, T. Ji, Y. Jiang, T. Lin, X. Zhou, and Z. Lin, "Efficient and generalizable cross-patient epileptic seizure detection through a spiking neural network," *Front. Neurosci.*, vol. 17, Art. 1303564, 2024.

[5] A. N. Pisarchik, N. Peña Serrano, W. Escalante Puente de la Vega, and R. Jaimes-Reátegui, "Hypergraph analysis of functional brain connectivity," preprint, Preprints.org.

[6] M. Ronzani and C. Silvano, "A case for hypergraphs to model and map SNNs on neuromorphic hardware," arXiv:2601.16118, 2026.

[7] M. Ronzani and C. Silvano, "GPU-accelerated hypergraph partitioning and placement to map SNNs on neuromorphic hardware," arXiv:2609.07577, 2026.

[8] M. Tangermann *et al.*, "Review of the BCI Competition IV," *Front. Neurosci.*, vol. 6, Art. 55, 2012, doi: 10.3389/fnins.2012.00055.

[9] C. Brunner, R. Leeb, G. Muller-Putz, and A. Schlogl, "BCI Competition 2008 – Graz data set A," Graz Univ. Technol., Graz, Austria, 2008.

[10] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Netw.*, vol. 10, no. 9, pp. 1659–1671, 1997.

[11] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, pp. 607–617, 2019.

[12] J. D. Nunes *et al.*, "Spiking neural networks: A survey," *IEEE Access*, vol. 10, pp. 60738–60764, 2022.

[13] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proc. IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[14] R. T. Schirrmeister *et al.*, "Deep learning with convolutional neural networks for EEG decoding and visualization," *Hum. Brain Mapp.*, vol. 38, no. 11, pp. 5391–5420, 2017.

[15] V. J. Lawhern *et al.*, "EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces," *J. Neural Eng.*, vol. 15, no. 5, Art. 056013, 2018.

[16] N. Kumar, G. Tang, R. Yoo, and K. P. Michmizos, "Decoding EEG with spiking neural networks on neuromorphic hardware," *Trans. Mach. Learn. Res.*, 2022.

[17] P. Tan, H. Xiao, H. Zou, K. Liu, Y. Shen, and C. Chen, "Spiking neural network classification method in motor imagery brain-computer interface," in *Proc. 19th Int. Conf. Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*, 2023, doi: 10.1109/ICNC-FSKD59587.2023.10281037.

[18] B. Schrauwen and J. Van Campenhout, "BSA, a fast and accurate spike train encoding scheme," in *Proc. IJCNN*, vol. 4, 2003, pp. 2825–2830.

[19] R. S. Banovoth and K. V. Kadambari, "Roman domination-based spiking neural network for optimized EEG signal classification of four class motor imagery," *Comput. Biol. Med.*, vol. 194, Art. 110397, 2025, doi: 10.1016/j.compbiomed.2025.110397.

[20] R. Ma, X. Lin, Z. Wang, X. Wang, and Y. Zhao, "Spiking neural networks with dual residual channel attention for decoding motor imagery EEG signals," *Biomed. Signal Process. Control*, vol. 117, Art. 109692, 2026, doi: 10.1016/j.bspc.2026.109692.

[21] X. Liao, Y. Wu, Z. Wang, D. Wang, and H. Zhang, "A convolutional spiking neural network with adaptive coding for motor imagery classification," *Neurocomputing*, vol. 549, Art. 126470, 2023.

[22] X. Liao, G. Li, Y. Wang, L. Sun, and H. Zhang, "Constructing lightweight and efficient spiking neural networks for EEG-based motor imagery classification," *Biomed. Signal Process. Control*, vol. 100, Art. 107000, 2025.

[23] Y. Li, L. Fan, H. Shen, and D. Hu, "HR-SNN: An end-to-end spiking neural network for four-class classification motor imagery brain-computer interface," *IEEE Trans. Cogn. Dev. Syst.*, vol. 16, no. 6, pp. 1955–1968, 2024.

[24] C. D. Virgilio G., J. H. Sossa A., J. M. Antelis, and L. E. Falcon, "Spiking neural networks applied to the classification of motor tasks in EEG signals," *Neural Netw.*, vol. 122, pp. 130–143, 2020.

[25] N. K. Kasabov, "NeuCube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data," *Neural Netw.*, vol. 52, pp. 62–76, 2014.

[26] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[27] F. Akopyan *et al.*, "TrueNorth: Design and tool flow of a 65 mW 1 million neuron programmable neurosynaptic chip," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1537–1557, 2015.

[28] S. B. Furber *et al.*, "The SpiNNaker project," *Proc. IEEE*, vol. 102, no. 5, pp. 652–665, 2014.

[29] D. Kudithipudi *et al.*, "Neuromorphic computing at scale," *Nature*, vol. 637, no. 8047, pp. 801–812, 2025.

[30] A. Balaji *et al.*, "Mapping spiking neural networks to neuromorphic hardware," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 1, pp. 76–86, Jan. 2020.

[31] S. Li *et al.*, "SNEAP: A fast and efficient toolchain for mapping large-scale spiking neural network onto NoC-based neuromorphic platform," in *Proc. ACM Great Lakes Symp. VLSI*, 2020, pp. 9–14.

[32] J. Xue *et al.*, "EdgeMap: An optimized mapping toolchain for spiking neural network in edge computing," *Sensors*, vol. 23, no. 14, Art. 6548, 2023.

[33] O. Jin *et al.*, "Mapping very large scale spiking neuron network to neuromorphic hardware," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 3, 2023, pp. 419–432.

[34] O. Jin *et al.*, "Mapping large-scale spiking neural network on arbitrary meshed neuromorphic hardware," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 11, pp. 2325–2340, 2025.

[35] Q. Xing *et al.*, "SNNcut: An efficient partitioning method for large-scale spiking neural networks using spike-sharing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2025, doi: 10.1109/TCAD.2025.3648616.

[36] P. K. Huynh, M. L. Varshika, A. Paul, M. Isik, A. Balaji, and A. Das, "Implementing spiking neural networks on neuromorphic architectures: A review," arXiv:2202.08897, 2022.

[37] L. Gottesburen *et al.*, "Scalable high-quality hypergraph partitioning," *ACM Trans. Algorithms*, vol. 20, no. 1, Art. 9, 2024.

[38] U. V. Catalyurek *et al.*, "More recent advances in (hyper)graph partitioning," *ACM Comput. Surv.*, vol. 55, no. 12, Art. 253, 2023.

[39] V. Jayaram and A. Barachant, "MOABB: Trustworthy algorithm benchmarking for BCIs," *J. Neural Eng.*, vol. 15, no. 6, Art. 066011, 2018, doi: 10.1088/1741-2552/aadea0.

[40] H. He and D. Wu, "Transfer learning for brain-computer interfaces: A Euclidean space data alignment approach," *IEEE Trans. Biomed. Eng.*, vol. 67, no. 2, pp. 399–410, Feb. 2020, doi: 10.1109/TBME.2019.2913914.